

Operational Game Semantics for generative algebraic effects

JAAFAR Hamza JABER Guilhem

^a INRIA Gallinette, Nantes, France

Abstract

We present a *sound* operational game semantics model (*w.r.t.* contextual equivalence) of a typed language with algebraic effects and handlers and dynamic generation of effect instances *à la* [5,11]. We exhibit the interactive aspect of effect propagation, and to address it precisely, we identify the adequate granularity of the operational semantics (*akin* to [13]) and the decomposition of normal forms into their interactive and their observational part. To account for this additional form of interaction, we extend the standard *pure* interaction interface of game semantics consisting of questions and answers with *effectful* moves that involve the propagation of effects and the yielding of *delimited* continuations. Finally, we extend the well-bracketed constraint on the behavior of the environment, from the use of continuations as is standard in game semantics, to fragments of captured delimited continuations.

Keywords: Algebraic effects and handlers, Game semantics

1 Introduction

Algebraic effects were originally introduced by Power and Plotkin [28,29] to provide a direct denotational account for syntactic entities that *perform* effects such as **get**, **set** for global state, **raise** for exceptions, or **chose** for non-deterministic choice, *etc.*

The combination of algebraic operations with effect handlers [30] is a more recent and powerful abstraction for programming with effects. Effect handlers generalize exception handlers by giving access not only to the raised effect but also to its delimited continuation. This enables a modular and compositional approach to effectful programming, supports user-defined effects, and facilitates effect combination in a seamless way.

A central challenge with this abstraction lies in managing multiple occurrences of the same effect while preserving modularity. Two main approaches have been proposed: the *lexical approach* and the *generative approach*, as described in [11]. In the lexical approach [31,8,9], one introduces effect instances locally, and the handler responsible for these instances is determined statically, preventing instance leakage. This static discipline enforces a form of *effect safety*, ensuring that all performed effects are handled. In contrast, the generative approach provides the possibility of generating dynamically fresh instances of an effect [5], that can be referred by their names, and be passed around as values. While this increases flexibility, it introduces challenges such as aliasing between names of effect instances, that needs to be tamed in order to enforce effect safety via a type-and-effect-system [11].

In this paper, we adopt the generative approach, but allow for *unsafe interaction*: effect instances may remain unhandled, and aliasing is not prevented. Our goal is to model the interaction between a program and its environment in such a language. Trace models for higher-order languages capture program behavior as sequences of observable actions -calls and returns- where the arguments of these actions may themselves

be (abstractions of) functions. We follow the methodology of *operational game semantics* (OGS) to define such a trace model.

The central idea of OGS is to represent programs as labelled transition systems (LTS), derived from the operational semantics of the language, in combination with a decomposition of normal forms into observable and interacting parts. This approach is particularly well suited to languages with generative effects, as demonstrated by Laird’s seminal work on ML-style higher-order references [20]. OGS also naturally accommodates control operators such as `call/cc` or the μ -binder of the $\lambda\mu$ -calculus [16]. In the absence of such control features, it enforces a *well-bracketing* discipline on the interaction—matching calls with returns—as is standard in game semantics [2, 15].

To extend OGS to a language with algebraic effect handlers, we incorporate:

- actions representing the execution of an effect;
- a fine-grained representation of the control flow between the program and its environment, to account for the exchange of delimited continuations triggered by handled effects.

Our long-term objective is to obtain a fully abstract trace model, capturing contextual equivalence of our language. As a first step in this direction, we prove that trace equivalence of our model is *sound* with respect to contextual equivalence. Moreover, we show that a well-bracketing condition can be imposed on the environment’s interaction without compromising soundness. However, we exhibit counterexamples to full abstraction: trace equivalence turns out to be too discriminating to characterize contextual equivalence.

To keep the model simple, we forbid the exchange of effect instances between the program and its environment. This avoids the need to dynamically track which effect instances have been disclosed—information that would otherwise be required for reasoning about interactions.

2 The language Λ_{eff}

2.1 Syntax of Λ_{eff}

We consider a fine-grained call-by-value λ -calculus [23] with typed algebraic effects and handlers. Its syntax is given in Figure 1 and its type system in Figure 2.

The computations of the shape $\iota\#\text{op}\mathbf{v}$ are responsible for triggering effects. They are given by an operation symbol op associated to an effect $\mathbb{E}$ and a particular instance ι of this effect. An effect $\mathbb{E}$ is described by its signature $\{(\text{op}_i : \tau_i \twoheadrightarrow \sigma_i)_i\}$. Λ_{eff} is also equipped with the syntactic construction $\text{new } \mathbb{E}$ responsible for the generation of a *fresh* instance of an effect of type $\mathbb{E}$. We stress that these instances are first-class values that can be exchanged in the same way exceptions or ML-style references, in say OCaml, could be shared.

For a handler $H = \uplus_i (\{\iota_i\#\text{op}_i x_i \kappa_i \mapsto u_i\})_i \uplus \{\text{ret } x \mapsto \mathbf{t}\}$, we define $H^{\text{return}} := \{\text{return } x \mapsto \mathbf{t}\}$ and $H^{\iota_i\#\text{op}_i} := \{\iota_i\#\text{op}_i x_i \kappa_i \mapsto u_i\}$, and $\text{hdl}(H) := \{(\iota_i\#\text{op}_i)_i\}$ (i.e. the exact set of effects handled by H).

In Fig. 1, we only highlight the additional names that would be absent in a standard presentation. They are given by *function names* $\mathcal{L}$ whose elements we range over by f, g , *delimited* and *undelimited continuation names* range over by $\kappa, \nu \in \mathcal{K}$ and $c, d \in \mathcal{C}$ respectively, as well as *effect names* $\mathbf{e} \in \mathcal{E}$. Further details about their semantic role will be postponed to later sections.

2.2 Operational semantics

The propagation of effects to abstract continuations is to be seen as a form of interaction (in a *game semantics* sense) whereby the *program* inquires whether the *environment*’s code (corresponding to the abstract continuation) handles some specific effect or not. For this reason, we privilege in Fig. 3 an abstract machine presentation of the *operational semantics* (à la [13]) whose reduction rules make this flow of information explicit through the *structural rules* $\mapsto_{\text{struct}} := \mapsto_{\text{fwd}} \cup \mapsto_{\text{push}}$ and the structure of its configurations.

The *reduction relation* $\mapsto_{\text{op}}$ is defined over pairs $(M \mid \Sigma)$ formed by a *running computation* M and an *effect instance context* Σ crucial to provide a semantics to $\text{new } \mathbb{E}$. The former has the shape $\langle \mathbf{t} \mid \mathbf{S} \circ \mathbf{T} \rangle$ consisting of three components: the *active term* $\mathbf{t}$, the *active evaluation stack* $\mathbf{S}$, and the *forwarded evaluation stack* $\mathbf{T}$. We will often use $\langle \mathbf{t} \mid \mathbf{S} \rangle$ as *syntactic sugar* for $\langle \mathbf{t} \mid \mathbf{S} \circ [] \rangle$ and conflate a term $\mathbf{t}$ with $\langle \mathbf{t} \mid [] \rangle$, i.e. its

types	$\tau, \sigma ::= \mathbb{1} \mid \mathbb{B} \mid \mathbb{Z} \mid \tau \times \sigma \mid \tau \rightarrow \sigma \mid \mathbb{E}$	
signatures $\mathbb{E}$	$::= \{(\mathbf{op}_i : \tau_i \rightarrow \sigma_i)_i\}$	
	(a) types	
values $v, w ::= () \mid \mathbf{false} \mid \mathbf{true} \mid n \mid \iota$		
	$\mid x \mid \lambda x. \mathbf{t} \mid \boxed{f}$	handlers $H ::= \{\mathbf{ret} x \mapsto \mathbf{t}\} \mid \{v \# \mathbf{op} x y \mapsto \mathbf{t}\} \uplus H$
terms $\mathbf{t}, u ::= \mathbf{return} v \mid v \# \mathbf{op} w \mid \boxed{\mathbf{e}} \mid \mathbf{new} \mathbb{E}$		eval frames $E ::= \mathbf{let} x = [] \mathbf{in} \mathbf{t} \mid \{[]\} \mathbf{with} H$
	$\mid w v \mid \mathbf{if} v \mathbf{then} \mathbf{t} \mathbf{else} u$	forward frames $F ::= [] \mid F[E]$
	$\mid S[\mathbf{t}]$	stack frames $S, T ::= [] \mid \boxed{\alpha}[] \mid T[S]$
	(b) expressions	
	$M, N ::= \langle \mathbf{t} \mid S \circ T \rangle \in \overline{\Lambda}_{\text{eff}}$	
	(c) running computations	

 Fig. 1. Λ_{eff} syntax.

initial embedding in running computations. We will also write $\Rightarrow_{\text{eval}}$ for $\mapsto_{\text{eval}} \cup \mapsto_{\text{push}}$.

The only non-standard rules are the starred ones ($\star_1$) and ($\star_2$) involving the propagation of effect in presence of *abstract codata*. Notice that we did not add the unsound rule ($\times$) which may seem like the innocuous dual to ($\star_1$).

$$\langle \iota \# \mathbf{op} v \mid S[\kappa[]] \circ T \rangle \mapsto_{\text{fwd}} \langle \iota \# \mathbf{op} v \mid S \circ \kappa[T] \rangle \quad (\times)$$

Indeed, as it will be evident in section 3, the ability to capture delimited continuations means that the *environment* can capture fragments of the *program's* evaluation stack, and as a result, gain control of a *program's* handler. Therefore, the possibility of $\kappa[]$ handling the effect (even though $\iota \# \mathbf{op} v$ is abstract from the environment's perspective) cannot *a priori* be ruled out.

2.3 Metatheory

2.3.1 Interactive Expressions

In the following, we will mostly be interested in the *interactive* subset of terms and computations on which our OGS semantics operate.

Definition 2.1 A term $\mathbf{t}$ (resp. a running computation M) is said to be *interactive* when it is well-typed and admits a typing of the form $\Sigma; \Gamma; \emptyset \vdash_{\text{c}} \mathbf{t} : \perp$ (resp. $\Sigma; \Gamma \Vdash_{\text{c}} M$).

2.3.2 Normal forms

Since *normal forms* describe the interaction interface between the program and its environment, we will privilege a useful presentation where we identify the *interaction patterns* that are causing a computation to be stuck. By expliciting these patterns, the normal forms are written as $P(\alpha)$; they involve a *co-pattern* P (described in fig. 5) representing the program controlled code and a *name* α abstracting an environment controlled codata. The token $\square$ represents a *codata hole* whose *filling*, i.e. the syntactic substitution thereof, is given by the application $(P, x) \mapsto P(x)$ of *co-patterns* to *codata* defined as $P(x) := P\{\square \mapsto x\}$.

Borrowing from the terminology of [7], we identify in fig. 4 *open-stuck* terms in which the reduction of the term depends on an abstract function f or an abstract delimited continuation $\kappa[]$ and where the continuation is controlled by the program, as well as *context-stuck* terms corresponding to unhandled effects and fully-evaluated returners inside an abstract continuation $c[]$. The two classes of normal forms correspond to different forms of interaction; the former calls for a *question* whereas the latter for an *answer*.

Lemma 2.2 (normal forms shape) *For all interactive computations $\Sigma; \Gamma \Vdash_{\text{c}} M$ such that $(M \mid \Sigma) \not\mapsto_{\text{op}}$, there exists a co-pattern P and a name $\alpha \in \text{dom}(\Gamma)$ s.t. $M = P(\alpha)$.*

$$\begin{array}{c}
\text{VAR} \frac{\Theta(x) = \tau}{\Sigma; \Gamma; \Theta \vdash_v x : \tau} \quad \text{I}_{\perp} \frac{}{\Sigma; \Gamma; \Theta \vdash_v () : \perp} \quad \text{I}_{\text{true}} \frac{}{\Sigma; \Gamma; \Theta \vdash_v \text{true} : \mathbb{B}} \quad \text{I}_{\text{false}} \frac{}{\Sigma; \Gamma; \Theta \vdash_v \text{false} : \mathbb{B}} \\
\\
\text{I}_{\rightarrow} \frac{\Sigma; \Gamma; \Theta, x : \tau, \Theta' \vdash_c t : \sigma}{\Sigma; \Gamma; \Theta, \Theta' \vdash_v \lambda x. t : \tau \rightarrow \sigma} \quad \text{E}_{\rightarrow} \frac{\Sigma; \Gamma; \Theta \vdash_v w : \tau \rightarrow \sigma \quad \Sigma; \Gamma; \Theta \vdash_v w : \tau}{\Sigma; \Gamma; \Theta \vdash_c w v : \sigma} \quad \text{I}_{\text{ret}} \frac{\Sigma; \Gamma; \Theta \vdash_v v : \tau}{\Sigma; \Gamma; \Theta \vdash_c \text{return } v : \tau} \\
\\
\frac{\left(\Sigma; \Gamma; \Theta, x_i : \tau_i, \kappa_i : \sigma_i \rightarrow \tau, \Theta' \vdash_c t_i : \sigma \quad \mathbb{E}_i(\text{op}_i) = \tau_i \Rightarrow \sigma_i \quad \Gamma \vdash v_i : \mathbb{E}_i \right)_{i \in I} \quad \Sigma; \Gamma; \Theta, x : \tau, \Theta' \vdash_c t : \sigma}{\Sigma; \Gamma; \Theta, \Theta' \vdash_h \{\text{ret } x \mapsto t\} \uplus \{v_i \# \text{op}_i x_i \kappa_i \mapsto t_i\}_i : \tau \Rightarrow \sigma} \text{HANDLER} \\
\\
\text{E}_{\text{handle}} \frac{\Sigma; \Gamma; \Theta \vdash_c t : \tau \quad \Sigma; \Gamma; \Theta \vdash_h H : \tau \Rightarrow \sigma}{\Sigma; \Gamma; \Theta \vdash_c \{t\} \text{ with } H : \sigma} \quad \text{E}_{\text{let}} \frac{\Sigma; \Gamma; \Theta \vdash_c t : \tau \quad \Sigma; \Gamma; \Theta, x : \tau, \Theta' \vdash_c u : \sigma}{\Sigma; \Gamma; \Theta, \Theta' \vdash_c \text{let } x = t \text{ in } u : \sigma} \\
\\
\text{E}_{\text{FF}} \frac{\Sigma(\text{op}) = \tau \Rightarrow \sigma \quad \Sigma; \Gamma; \Theta \vdash_v v : \tau}{\Sigma; \Gamma; \Theta \vdash_c \iota \# \text{op } v : \sigma} \quad \text{INST} \frac{\Sigma(\iota) = \mathbb{B}}{\Sigma; \Gamma; \Theta \vdash_v \iota : \mathbb{B}} \quad \text{NEW} \frac{}{\Sigma; \Gamma; \Theta \vdash_c \text{new } \mathbb{E} : \mathbb{E}} \\
\text{(a) type judgements for expressions} \\
\\
\text{HOLE} \frac{}{\Sigma \Vdash_s [] : (\Theta \vdash \tau) \rightsquigarrow (\Theta \vdash \tau)} \\
\\
\frac{\Sigma \Vdash_s T : (\Theta \vdash \tau) \rightsquigarrow (\Xi \vdash \sigma) \quad \Sigma; \Theta \vdash_h H : \sigma \Rightarrow \varphi}{\Sigma \Vdash_s \{T\} \text{ with } H : (\tau \vdash \Theta) \rightsquigarrow (\varphi \vdash \Xi)} \quad \frac{\Sigma \Vdash_s T : (\Theta, \Theta' \vdash \sigma) \rightsquigarrow (\Xi \vdash \tau) \quad \Sigma; \Theta, x : \tau, \Theta' \vdash_c t : \varphi}{\Sigma \Vdash_s \text{let } x = T \text{ in } t : (\Theta, x : \tau, \Theta' \vdash \sigma) \rightsquigarrow (\Xi \vdash \varphi)} \\
\\
\frac{\Sigma \Vdash_s K : (\Xi \vdash \sigma) \rightsquigarrow (\Psi \vdash \varphi) \quad \Sigma \Vdash_s T : (\Theta \vdash \tau) \rightsquigarrow (\Xi \vdash \sigma) \quad \Sigma; \Gamma; \Theta \vdash_c t : \tau}{\Sigma; \Gamma; \Psi \vdash_c \langle t \mid K \circ T \rangle : \varphi} \\
\text{(b) type judgements for evaluation stacks and running computations} \\
\\
\text{FNAME} \frac{\Gamma(f) = \tau}{\Sigma; \boxed{\Gamma}; \Theta \vdash_v f : \tau} \quad \text{EFFNAME} \frac{e \in \text{dom}(\Gamma)}{\Sigma; \boxed{\Gamma}; \Theta \vdash_c e : ?} \\
\\
\text{CONTNAME} \frac{\Gamma(\alpha) = \tau \rightsquigarrow \sigma}{\Sigma, \Gamma \Vdash_s \alpha [] : (\Theta \vdash \tau) \rightsquigarrow (\Theta \vdash \sigma)} \\
\\
\frac{\Sigma, \Gamma \Vdash_s T : (\Theta \vdash \tau) \rightsquigarrow (\Xi \vdash \sigma) \quad \Sigma; \Gamma; \Theta \vdash_c t : \tau}{\Sigma; \Gamma; \Xi \vdash_c T[t] : \sigma} \\
\\
\frac{\Sigma, \Gamma \Vdash_s K : (\Xi \vdash \sigma) \rightsquigarrow (\emptyset \vdash \perp) \quad \Sigma, \Gamma \Vdash_s T : (\Theta \vdash \tau) \rightsquigarrow (\Xi \vdash \sigma) \quad \Sigma; \Gamma; \Theta \vdash_c t : \tau}{\boxed{\Sigma; \Gamma \Vdash_c \langle t \mid K \circ T \rangle}} \\
\text{(c) typing of names and interactive terms.}
\end{array}$$

Fig. 2. Type System

Our notion of program equivalence builds on the *Closed Instantiations of Uses* (CIU) equivalence, introduced by Mason and Talcott [24]. Whereas Morris-style contextual equivalence [25] quantifies over general contexts—including those placing holes under λ -abstractions to handle open terms—CIU-equivalence restricts attention to evaluation contexts and handles open terms by quantifying over substitution maps as well.

In our setting, we refine this idea by leveraging the distinction between variables and names: we define equivalence only for interactive terms $\Sigma; \Gamma \vdash_c M$, where M contains no free variables, and all names are drawn from Γ . We assume that Γ contains exactly one continuation name, no delimited continuation names, and that all types in Γ are free of effect signatures $\mathbb{E}$. This last restriction ensures that no effect

$$\begin{aligned}
& \langle (\lambda x. t) v \mid S \mid \Sigma \rangle \mapsto_{\text{eval}} \langle t\{x := v\} \mid S \mid \Sigma \rangle \\
& \langle \text{new } \mathbb{E} \mid S \mid \Sigma \rangle \mapsto_{\text{eval}} \langle \text{return } \iota \mid S \mid \Sigma \cdot [\iota \mapsto \mathbb{E}] \rangle \\
& \langle \text{return } v \mid S[\text{let } x = \square \text{ in } t] \mid \Sigma \rangle \mapsto_{\text{eval}} \langle t\{x := v\} \mid S \mid \Sigma \rangle \\
& \langle \text{return } v \mid S[\{\square\} \text{ with } H] \mid \Sigma \rangle \mapsto_{\text{eval}} \langle t\{x := v\} \mid S \mid \Sigma \rangle \quad \text{when } H^{\text{ret}} = \{\text{return } x \mapsto t\} \\
& \langle \iota \# \text{op } v \mid S[\{\square\} \text{ with } H] \circ T \mid \Sigma \rangle \mapsto_{\text{eval}} \langle t\{x := v\}\{y := \lambda z. \{T[\text{return } z]\} \text{ with } H\} \mid S \mid \Sigma \rangle \\
& \quad \text{when } H^{\text{op}} = \{\iota \# \text{op } x y \mapsto t\} \\
& \langle \iota \# \text{op } v \mid S[E] \circ T \mid \Sigma \rangle \mapsto_{\text{fwd}} \langle \iota \# \text{op } v \mid S \circ E[T] \mid \Sigma \rangle \quad \text{when } \iota \# \text{op} \notin \text{hdl}(E) \\
& \langle e \mid S[E] \circ T \rangle \mapsto_{\text{fwd}} \langle e \mid S \circ E[T] \rangle \quad (\star_1) \\
& \langle E[t] \mid S \mid \Sigma \rangle \mapsto_{\text{push}} \langle t \mid S[E] \mid \Sigma \rangle \\
& \langle \alpha[t] \mid S \rangle \mapsto_{\text{push}} \langle t \mid S[\alpha[\square]] \rangle \quad (\star_2)
\end{aligned}$$

Fig. 3. operational semantics

$$\begin{aligned}
\Lambda_{\text{eff}}^{\text{nf}} \ni \text{Nf} ::= & \langle \text{return } v \mid c[\square] \rangle \mid \langle \text{return } v \mid S[\kappa[\square]] \rangle \quad \text{evaluated returners} \\
& \mid \langle \underline{e} \mid S[\kappa[\square]] \circ T \rangle \mid \langle \underline{e} \mid c[\square] \circ T \rangle \quad \text{unhandled effects} \\
& \quad \text{where } \underline{e} ::= e \mid \iota \# \text{op } v \\
& \mid \langle f v \mid S \rangle \quad \text{open applications}
\end{aligned}$$

Fig. 4. interactive normal forms of Λ_{eff} .

$$\begin{aligned}
P ::= & \langle \text{return } v \mid \square \rangle \mid \langle \underline{e} \mid \square \circ S \rangle \quad \text{context-stuck} \\
& \mid \langle \text{return } v \mid S[\square] \rangle \mid \langle \underline{e} \mid S[\square] \circ T \rangle \} \quad \text{open-stuck} \\
& \mid \langle \square v \mid S \rangle \\
& \text{where } \underline{e} ::= e \mid \iota \# \text{op } v \\
& \quad \text{(a) syntax of co-patterns}
\end{aligned}$$

$$\begin{array}{c}
\text{CONT-VAL} \quad \frac{\Sigma; \Gamma \vdash_v v : \tau}{\Sigma; \Gamma \vdash \langle \text{return } v \mid \square \rangle : \neg \neg \tau} \quad \text{CONTROL-STUCK} \quad \frac{\Sigma; \Gamma \vdash_c \underline{e} : \tau \quad \Sigma; \Gamma \vdash_s S : \tau \rightsquigarrow \sigma}{\Sigma; \Gamma \vdash \langle \underline{e} \mid \square \circ S \rangle : \neg \neg \sigma} \\
\text{LAMBDA} \quad \frac{\Sigma; \Gamma \vdash_v v : \tau \quad \Sigma; \Gamma \vdash S : \neg \sigma}{\Sigma; \Gamma \vdash \langle \square v \mid S \rangle : \neg(\tau \rightarrow \sigma)} \quad \text{OPEN-STUCK} \quad \frac{\text{DELIM-CONT} \quad \Sigma; \Gamma \vdash P : \neg \neg \tau \quad \Sigma; \Gamma \vdash S : \neg \sigma}{\Sigma; \Gamma \vdash P(S[\square]) : \neg(\tau \rightsquigarrow \sigma)}
\end{array}$$

(b) typing judgements for co-patterns

Fig. 5. co-patterns: syntax and types.

instances can be exchanged between the program and its environment. We refer to such typing contexts as initial, since these constraints do not generally hold as interaction proceeds. Equivalence between M_1 and M_2 can then be defined as each approximating the other. Quantifying over name assignment for Γ , the continuation name d of Γ is then mapped to an evaluation context. This provides a definition of CIU equivalence quantifying solely on name assignment.

Definition 2.3 (ciu-approximation) We consider two interactive terms M_1, M_2 such that both $\Sigma; \Gamma \Vdash_c M_i$ (for $i \in \{1, 2\}$), with Γ initial. Then M_1 is said to be *ciu-approximated* by M_2 , written $\Sigma; \Gamma \vdash M_1 \preceq_{\text{ciu}} M_2$, when for all instance contexts $\Sigma' \supseteq \Sigma$, for all continuation names c_f and for all name assignments γ such

that $\Sigma'; c_f : \neg 1 \vdash \gamma : \Gamma$, if $(M_1\{\gamma\} \mid \Sigma') \Downarrow_{\text{op}} c_f[()]$ then $(M_2\{\gamma\} \mid \Sigma') \Downarrow_{\text{op}} c_f[()]$.

Equivalence between M_1 and M_2 is then defined via mutual approximation. Quantifying over all name assignments for Γ , the continuation name d is interpreted as an evaluation context. This yields a form of CIU-equivalence based solely on quantification over name assignments.

We note that this definition excludes terms with names of positive types such as $\mathbb{B}$ or $\mathbb{Z}$, which should rather be handled using free variables. A mild generalization of our approach could accommodate such terms by also quantifying over variable substitutions. However, for the sake of simplicity, we leave this extension to future work.

3 Abstract Interaction

The abstract interaction LTS is a bipartite labelled transition system. It describes the *interactive execution* of a term (*Proponent*) inside and abstract *program context* (*Opponent*). Throughout the execution, both parties play the role of an *active term* as well as that of an *active evaluation stack*, which is reflected in *Proponent*'s alternating between *active* and *passive* states, hence the bipartite part. The transitions mark this interactive role swapping and their corresponding labels correspond to the active *Player's* move. The internal operational evaluation of *Proponent* (in an active state) are accounted for by silent transitions.

3.1 Nominal abstraction

When an *active term* reaches a *normal form*, it discloses the information needed for the *environment* to continue computing a value. This is done through the *abstraction process* whereby the *codata* occurring in the active *normal form* are abstracted away and transformed into *names* that are handed over to the *environment* as part of various *nominal patterns*, capturing the fact that the latter's behaviour can only be parametric *w.r.t.* these *codata* (unlike *data* that could be inspected to trigger a *data-dependent* reaction).

In a dual manner, upon receiving a *nominal pattern*, the *passive* player (who knows its type) infers the types of the underlying disclosed names through a *type inference system*.

In the following, we will define the *nominal abstraction* of a few syntactic categories, where we will introduce, for each, their *abstract* counterpart, the *type inference system* as well as the generating *focussing process* that transforms a *syntactic object* into an ultimate pattern [22] (its positive skeleton) together with a corresponding name assignment map (its negative filling).

3.1.1 Abstract values

The values exchanged between *Proponent* and *Opponent* are abstracted away, by transforming a value into a *nominal ultimate pattern* (nup) A defined over a set of *function names* $f \in \mathcal{L}$:

$$V, W := () \mid n \mid \text{false} \mid \text{true} \mid f \mid \langle V, W \rangle$$

We define in fig. 6 the *abstraction process* of values, where $\nearrow$ denotes the corresponding *focussing process*.

3.1.2 Abstract generalised values

Not only the structure of values is exposed in the interaction of *Opponent* and *Proponent*, but so are some fully-evaluated computations. These *generalised values*¹ are in turn abstracted away into a *nominal computation pattern* (ncp) which is defined as:

$$\underline{A}, \underline{B} := \text{ret } A \mid \mathbf{e} \mid r[\mathbf{e}]$$

where $\mathbf{e} \in \mathcal{E}$ is an abstract *effect name* and $r \in \text{List}(\mathcal{K})$ is a *abstract (delimited) evaluation stack* defined over the set $\mathcal{K}$ of *delimited continuations names*.

Analogously to abstract values, we lift the *focusing process* to one that transforms a computation in normal form into an abstract computation. We will denote it by the symbol $\nearrow$.

¹ computations that *are* but cannot *do*.

$$\begin{array}{c}
 \frac{\emptyset \vdash_v v : \tau \quad \tau \in \{\mathbb{1}, \mathbb{Z}, \mathbb{B}\}}{v \nearrow (v, \varepsilon)} \quad \frac{}{\lambda x. t \nearrow (f; [f \mapsto \lambda x. t])} \quad \frac{v \nearrow (V; \gamma) \quad w \nearrow (W; \gamma')}{\langle v, w \rangle \nearrow (\langle V, W \rangle, \gamma \cdot \gamma')} \quad \frac{}{f \nearrow (g; [g \mapsto f])} \\
 \\
 \frac{\emptyset \vdash_v v : \tau \quad \tau \in \{\mathbb{1}, \mathbb{Z}, \mathbb{B}\}}{\Gamma \Vdash_v v : \tau \triangleright \emptyset} \quad \frac{}{\Gamma \Vdash_v f : \tau \rightarrow \sigma \triangleright [f \mapsto (\tau \rightarrow \sigma)]} \\
 \text{(a) abstraction of values.} \\
 \\
 \frac{v \nearrow (A; \gamma)}{\text{return } v \nearrow (\text{ret } A; \gamma)} \quad \frac{}{\iota \# \text{op } v \nearrow (e; [e \mapsto \iota \# \text{op } v])} \\
 \\
 \frac{\Gamma \Vdash_v A : \tau \triangleright \Delta}{\Gamma \Vdash_c \text{ret } A : \tau \triangleright \Delta; \emptyset} \quad \frac{}{\Gamma \Vdash_c e : ? \triangleright [e \mapsto ?]; \emptyset} \quad \frac{\Gamma \vdash_c r[e] : \tau}{\Gamma \Vdash_c \kappa :: r[e] : \sigma \triangleright \emptyset; [\kappa \mapsto \tau \rightsquigarrow \sigma]} \\
 \text{(b) abstraction of generalised values.}
 \end{array}$$

Fig. 6. nominal abstraction process.

3.1.3 Abstract normal forms

Continuing with our nominal abstraction of expressions, we introduce *abstract normal forms* and *abstract copatterns* (acp) and for the sake of a uniform treatment in-keeping with the classification of copatterns of fig. 5, we will express them in terms of *abstract computations* in order to exhibit the underlying *generalized* questions and answers structure.

$$\begin{array}{ll}
 \text{CoPatterns} \ni p, q ::= \langle \underline{A} \mid \square \rangle & \text{context-stuck} \\
 \mid \langle \square \underline{A} \mid d \rangle \mid \langle \square A \mid d \rangle & \text{open-stuck}
 \end{array}$$

where $\langle r[e] \mid \square \rangle$, $\langle \square r[e] \mid d \rangle$ and $\langle \square \text{ret } A \mid d \rangle$ are *syntactic sugar* for $\langle e \mid \square \circ r \rangle$, $\langle e \mid d[\square] \circ r \rangle$ and $\langle \text{return } A \mid d[\square] \rangle$ respectively, and where the *hole filling* is given by the application $(p, \alpha) \mapsto p(\alpha)$ where $p(\alpha) := p\{\square := \alpha\}$.

In fig. 7 we introduce the abstraction process of *copatterns*. Naturally, it produces an *abstract copattern*, a name assignment γ , in addition to a component ξ , the *abstract forward*, carrying information about the *forward evaluation stack*. It is either of the shape $\emptyset$ (indicating the absence of effects) or (e, r, δ) (indicating a performed effect e to be forwarded or handled) where r represents the *abstract (delimited) stack* through which said effect has been propagated and δ is another name assignment for abstracting the effects and forward evaluation frames.

Example 3.1 Perhaps the simplest example is a fully-evaluated returner in the form of $c[\text{return } v]$ that can be written as $(\square \text{return } v)(c)$ and whose abstraction can be expressed as an abstract copattern $p = \langle \text{ret } A \mid \square \rangle$ and the abstract codata c .

Remark 3.2 Notice how the abstraction process of normal forms is not total. It is only defined on effectful normal forms of a specific shape (involving *forward frames* F) that reflects the order of effect propagation and the incrementation of the *abstract forward* mirrors this gradual interactive process. The choice of keeping γ and δ (of ξ) and to combine abstract codata belonging to both the program and the environment in r has a technical benefit as well as a conceptual one; as it maintains the distinction between passive interactions and interactions with observation.

3.2 Abstract Interactive LTS

In the *abstract interaction*, the executing term (*Proponent*) will be represented by an LTS configuration in order to be evaluated against an *abstract environment* (*Opponent*) that models all compatible *program environments* while exhibiting the *interaction* between the two that gets obscured by plain *syntax substitution*. To this effect, the configurations will carry an *information* component that keeps track of the history of the *interaction*. It consists of a map between the names and the *concrete* code of codata that *Proponent* has disclosed and rendered accessible to *Opponent*.

$$\begin{array}{c}
 \text{CONTEXT-STUCK} \\
 \frac{v \nearrow (A, \gamma_A)}{\langle \text{return } v \mid \square \rangle \not\Rightarrow (\langle \text{ret } A \mid \square \rangle, \gamma_A, \emptyset)} \text{ RETURN} \\
 \\
 \frac{}{\langle \iota \# \text{op } v \mid \square \circ F \rangle \not\Rightarrow (\langle \kappa[e] \mid \square \rangle, \varepsilon, (e, \kappa, [e \mapsto \iota \# \text{op } v] \cdot [\kappa \mapsto F]))} \text{ PERFORM} \\
 \\
 \frac{}{\langle e \mid \square \circ F[r] \rangle \not\Rightarrow (\langle \kappa :: r[e] \mid \square \rangle, \varepsilon, (e, \kappa :: r, [\kappa \mapsto F]))} \text{ FORWARD} \\
 \\
 \text{OPEN-STUCK} \\
 \frac{v \nearrow (A, \gamma_A)}{\langle \square v \mid S \rangle \not\Rightarrow (\langle \square A \mid c \rangle, \gamma_A \cdot [c \mapsto S], \emptyset)} \quad \frac{C \not\Rightarrow (p, \gamma_C, \xi)}{C(S[\square]) \not\Rightarrow (p(c[\square]), \gamma_C \cdot [c \mapsto S], \xi)} \\
 \text{(a) abstracting copatterns} \\
 \\
 \text{CONTEXT-STUCK} \\
 \frac{\Gamma \Vdash_C \underline{A} : \tau \triangleright \Gamma_{\underline{A}}; \Delta_{\underline{A}}}{\Gamma \Vdash \langle \underline{A} \mid \square \rangle : \neg \neg \tau \triangleright \Gamma_{\underline{A}}; \Delta_{\underline{A}}} \\
 \text{OPEN-STUCK} \\
 \frac{\Gamma \Vdash_C \underline{A} : \tau \triangleright \Gamma_{\underline{A}}; \Delta_{\underline{A}}}{\Gamma \Vdash \langle \square \underline{A} \mid d \rangle : \neg(\tau \rightsquigarrow \sigma) \triangleright \Gamma_{\underline{A}} \cdot [d \mapsto \neg \sigma]; \Delta_{\underline{A}}} \quad \frac{\Gamma \Vdash_V A : \tau \triangleright \Gamma_A}{\Gamma \Vdash \langle \square A \mid d \rangle : \neg(\tau \rightarrow \sigma) \triangleright \Gamma_A \cdot [d \mapsto \neg \sigma]; \emptyset} \\
 \text{(b) copatterns typing rules.}
 \end{array}$$

Fig. 7. abstraction process of copatterns.

We define next the abstract interactive LTS $\mathcal{L}_{\text{AI}}$ that is given by $(\mathcal{A}, \mathcal{M}, \xrightarrow{\mathbf{m}}_1)$.

3.2.1 configurations, moves and transitions

$$\begin{array}{lll}
 \text{configurations} & \mathcal{A} \ni \mathbb{I}, \mathbb{J} ::= \langle \Sigma; \gamma; \xi \rangle & \text{active state} \\
 & | \langle (\mathbb{M} \mid \Sigma); \gamma; \delta \rangle & \text{passive state} \\
 \\
 \text{moves} & \mathcal{M} \ni \mathbf{m} := \alpha.p^\oplus \mid \alpha.p^\ominus &
 \end{array}$$

The moves are given by the previously described *abstract copatterns* where the action $\alpha.p$ is to be understood as the combination of an *interaction handle* α and an input p : the abstract codata α belonging to the *passive* player is being probed with the *active* player's abstract copattern p .

The superscript $\oplus$ (resp. $\ominus$) is a convenient notation to indicate that *Proponent* (resp. *Opponent*) is the *active* player performing the *move*. We will sometimes range over these polarized moves by $\mathbf{p}$ and $\mathbf{o}$ respectively.

Breaking down all the possible *interactive transitions* witnessed by $\alpha.p$, we have:

$$\begin{array}{ll}
 \mathbf{c}.\langle \text{ret } A \mid \square \rangle^\oplus & \text{returning a value to the abstract context } c[] \\
 \\
 \mathbf{c}.\langle r[e] \mid \square \rangle^\oplus & \text{propagating } e \text{ and the delimiting continuation } r \\
 & \text{to the abstract context } c[] \\
 \\
 \mathbf{f}.\langle \square A \mid d \rangle^\oplus & \text{requesting the result of the application } f A \text{ in } d[] \\
 \\
 \mathbf{\kappa}.\langle \square[\text{ret } A] \mid d \rangle^\oplus & \text{requesting the result of the computation } \kappa[\text{ret } A] \text{ in } d[] \\
 \\
 \mathbf{\kappa}.\langle \square[r[e]] \mid d \rangle^\oplus & \text{delegating the handling or propagation of } e \text{ with its} \\
 & \text{delimiting continuation } r \text{ to } \kappa \text{ and requesting the result in } d[]
 \end{array}$$

After a series of internal reduction steps $\mapsto_{\text{op}}^*$, the *active* configuration reaches a *normal form*. It

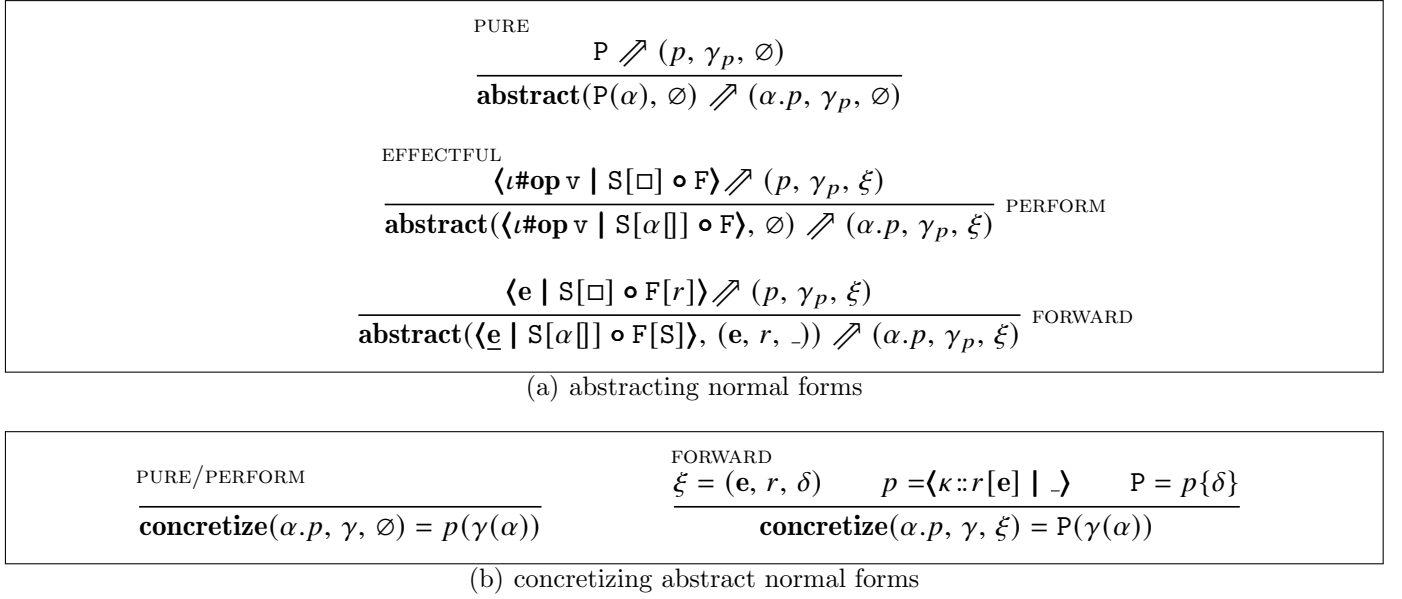


Fig. 8. the dualizing process.

performs a transition into an *passive state* where the *codata* in the corresponding copattern is *abstracted* then disclosed or forwarded to the environment via the information components.

A transition from a *passive configuration* admits *branching*. Indeed, since the *program environment* is abstract and not fixed, it can use any of the disclosed information to perform a *move* triggering a transition into an *active configuration* and the *concretization* of a running computation.

The transitions as well as the underlying abstraction and concretization processes are defined in fig. 9 and fig. 8, respectively.

We will sometimes write $\blacksquare \xi$ (and $\blacksquare \delta$), putting a modality $\blacksquare \in \{!, ?\}$ in front of this component to indicate who's the author of the effect. Accordingly, $!(e, _, \delta)$ means that *Proponent* performed the effect and that *Opponent can only forward* it ($e \in \text{dom}(\delta)$ is implied), while $?\delta$, on the other hand, reads as *Opponent may forward* the effect ($\text{dom}(\delta) \cap \mathcal{E} = \emptyset$ is implied in this case). When it is equal to $\emptyset$ we will often omit it altogether from configurations.

For a compact presentation, we will abuse the notation of maps and substitution on ξ when it is the constituting component δ that is intended instead.

Remark 3.3 As mentioned in section 2.2 where we have introduced $\mapsto_{\text{op}}$, the reduction $\mapsto_{\text{fwd}}$ captures a flow of information; that of effect propagation. A series of $\mapsto_{\text{fwd}}$ reductions not followed by an $\mapsto_{\text{eval}}$ reduction results in an unhandled effect forwarded to the environment, which in turn could potentially handle it, capturing the associated delimited continuation in the process.

This information flow is a form of interaction between the program and its environment consisting in inquiring whether the environment handles the effect or not. The abstract effect and its enclosing evaluation context are provisionally forwarded as a fragment of the "would-be" captured delimited continuation in case the environment catches the effect and handles it. In case it does not, a pseudo copy-cat action (forwarding back the same effect but with an extended evaluation context) is expected instead.

By way of example, we will give an informal presentation of the *interaction* of a term with a *concrete* program context, which will map to one possible *interactive execution path* against an *abstract environment* in $\mathcal{L}_{\text{AI}}$.

Example 3.4 We consider the interaction of the term $u := \text{let } y = \text{new } \mathbb{E} \text{ in } \{f(\lambda x. y \# \mathbf{op}())\} \text{ with } H^z$, where $H^z := \{\text{ret } x \mapsto \text{return } x\} \uplus \{z \# \mathbf{op} \, x \, \kappa \mapsto \kappa \, 5\}$, with the the program context $C := \text{let } f = (\lambda y. y \, v; \text{return true}) \text{ in } []$, that is in turn equivalent to the ciu-environment $E = ([], [f \mapsto \lambda g. g \, v; \text{return true}])$.

We will represent the term u by the *active* configuration $\langle d_0[u], \varepsilon \rangle$ and the environment E by the *passive* configuration $\langle \gamma_o \rangle$ where $\gamma_o = [d_0 \mapsto c_f[[]]] \cdot [f \mapsto \lambda x. x \, v; \text{return true}]$.

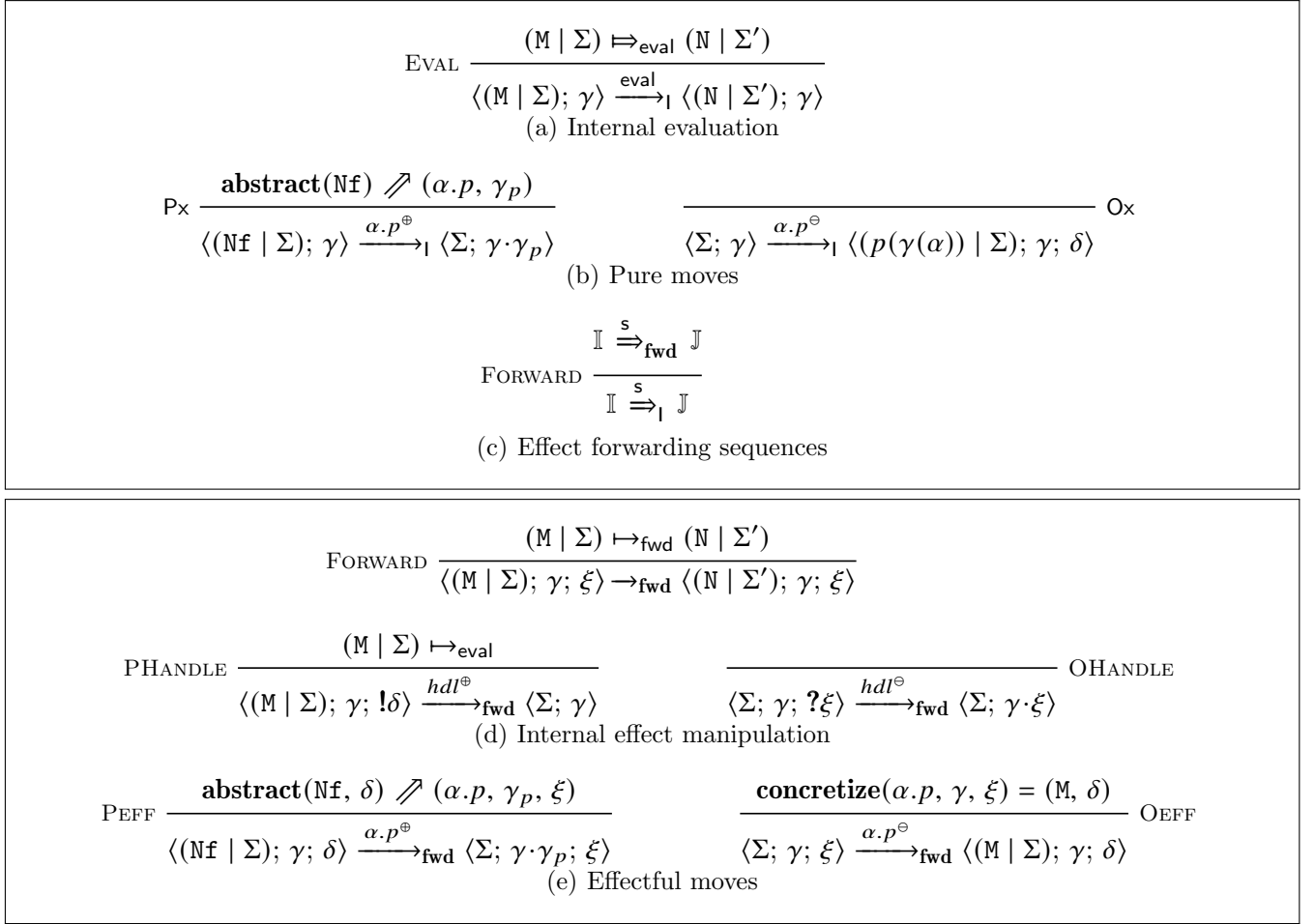


Fig. 9. transitions of the Abstract Interaction LTS

Reading from top to bottom, the figure. 10 illustrates their *concrete* interaction as well as its corresponding *path* in the *abstract interaction* LTS. After every interaction point, the active player plays a move and swaps roles with the *passive* player. The *codata* α that underlies the move is highlighted like so α .

3.3 Typing Constraints

Definition 3.5 *Type configurations* $\mathbb{T}; \mathbb{S}$ are of the shape $\langle \Gamma_o; \Delta_o \mid \Gamma_p; \Delta_p \rangle$ where:

- Γ_p, Γ_o are two *disjoint* type contexts for *Program names* and *Environment names*, respectively.
- Δ_p, Δ_o are two *disjoint* type contexts for effects and forwarded *abstract delimited continuations*.
- For $y \in \{o, p\}$. Γ_y and Δ_y are disjoint.

The typing judgements $\mathbb{E} \vdash \mathbb{T}$ and $\mathbb{P} \vdash \mathbb{S}$ denotes that the configurations $\mathbb{E}$ and $\mathbb{P}$ are of type $\mathbb{T}$ and $\mathbb{S}$ respectively. We define the corresponding typing relation fig. 11.

We will denote by $\mathbb{T}^\perp$ the type configuration dual to $\mathbb{T}$ defined by:

$$\langle \Gamma_o; \Delta_o \mid \Gamma_p; \Delta_p \rangle^\perp := \langle \Gamma_p; \Delta_p \mid \Gamma_o; \Delta_o \rangle$$

The *Type LTS* $\mathcal{L}_T$ is defined as $(\mathcal{T}; \mathcal{M}; \xrightarrow{\mathbf{m}}_T)$ with $\xrightarrow{\mathbf{m}}_T$ defined in figure 12, where $\mathcal{T}$ is the set of type configurations.

d_0, f	$\Vdash$	$d_0[\{f\}(\lambda x. \iota \# \text{op } ())] \text{ with } H'$	$f.\langle \square g \mid c_0 \rangle^\oplus$
c_0, g	$\Vdash$	$c_0[(\lambda y. y n; \text{return true}) g]$ $\Downarrow_{\text{eval}}$ $c_0[\overline{g} n; \text{return true}]$	$g.\langle \square n \mid d_1 \rangle^\ominus$
d_0, f, d_1	$\Vdash$	$d_1[(\lambda x. \iota \# \text{op } ()) n]$ $\Downarrow_{\text{eval}}$ $\overline{d_1}[\iota \# \text{op } ()]$	$d_1.\langle \kappa[e] \mid \square \rangle^\oplus$
$c_0, g! (e, \kappa)$	$\Vdash$	$c_0[\kappa[e]; \text{return true}]$ $\Downarrow_{\text{fwd}}$ $\langle e \mid \overline{c_0}[] \mid \kappa[\overline{[]}; \text{return true}] \rangle$	$c_0.\langle \kappa::\kappa[e] \mid \square \rangle^\ominus$
$d_0, f, d_1! (e, \kappa::\kappa)$	$\Vdash$	$\langle \iota \# \text{op } () \mid d_0[\{\overline{[]}\} \text{ with } H'] \mid \kappa[\overline{[]}] \rangle$ $\Downarrow_{\text{hdl}}$	$hdl^\oplus$
$d_0, f, d_1, [\kappa]$	$\Vdash$	$\langle (\lambda y. \{\kappa[\text{return } y]\} \text{ with } H') 5 \mid d_0[] \rangle$ $\Downarrow_{\text{eval}}$ $\langle \text{return } 5 \mid d_0[\{\overline{[]}\} \text{ with } H'] \rangle$	$\kappa.\langle \square[\text{ret } 5] \mid c_1 \rangle^\oplus$
c_0, g, c_1	$\Vdash$	$\langle \text{return } 5; \text{return true} \mid c_1[\overline{[]}] \rangle$ $\Downarrow_{\text{op}}$ $\overline{c_1}[\text{return true}]$	$c_1.\langle \text{ret true} \mid \square \rangle^\ominus$
$d_0, f, d_1, [\kappa]$	$\Vdash$	$d_0[\{\text{return true}\} \text{ with } H']$ $\Downarrow_{\text{eval}}$ $\overline{d_0}[\text{return true}]$	$d_0.\langle \text{ret true} \mid \square \rangle^\oplus$
c_0, g, c_1	$\Vdash$	return true	

 Fig. 10. effectful interaction of $\mathbf{u}$ and E .

PASSIVE CONF.	ACTIVE CONF.
$\text{dom}(\gamma) = \Gamma_p \quad \Gamma_o \vdash \gamma : \Gamma_p \quad \Gamma_p \vdash \xi : \Delta_o$	$\text{dom}(\gamma) = \Gamma_p \quad \Sigma; \Gamma_o \cdot \Delta_o \Vdash_c M \quad \Gamma_o \vdash \delta : \Delta_p$
$\langle \Sigma; \gamma; \xi \rangle \circ \langle \Gamma_o; \Delta_o \mid \Gamma_p; \Delta_p \rangle$	$\langle (M \mid \Sigma); \gamma; \delta \rangle \circ \langle \Gamma_o; \Delta_o \mid \Gamma_p; \Delta_p \rangle$

Fig. 11. configuration typing.

PACTION	
$\Gamma_o(\alpha) = \tau \quad \Gamma_o \cdot \Delta_p \cdot \Delta_o \Vdash q : \neg \tau \triangleright \Gamma_q; \Delta_q$	
$\langle \Gamma_o; \Delta_o \mid \Gamma_p; \Delta_p \rangle \xrightarrow{\alpha.q^\oplus}_{\mathcal{T}} \langle \Gamma_o; \Delta_o \mid \Gamma_p \cdot \Gamma_q; \Delta_p \cdot \Delta_q \rangle$	
PHANDLE	
$\langle \Gamma_o; \Delta_o \mid \Gamma_p; \Delta_p \rangle \xrightarrow{hdl^\oplus}_{\mathcal{T}} \langle \Gamma_o \cdot \Delta_o; \emptyset \mid \Gamma_p; \emptyset \rangle$	
(a) proponent transitions.	
	$\mathbb{T} \xrightarrow{\mathbf{m}}_{\mathcal{T}} \mathbb{S}$
	$\mathbb{T}^\perp \xrightarrow{\mathbf{m}^\perp}_{\mathcal{T}} \mathbb{S}^\perp$
	(b) symmetrical transitions.

 Fig. 12. Definition of $\mathcal{L}_{\mathcal{T}}$

3.4 Well-bracketing Constraints

The interactive configurations $\mathbb{I} \in \mathcal{A}$ we have considered thus far represent *Proponent's* one-sided perspective on the computation. The *information component* contains all of its *codata* that had been disclosed up to the current stage of the computation, namely the *continuations*. However, this component (and by extant any *passive configuration*), as it is defined, is history-agnostic whereas the language Λ_{eff} only permits a constrained usage of these *abstract continuations* that respects a certain history-sensitive *bracketing discipline*.

In this section, we define the well-bracketing LTS that addresses this issue by constraining the moves of *Opponent* in $\mathcal{L}_{\text{AI}}$, so that the generated *well-bracketed* behaviours are correspond to actual concrete interactions.

3.4.1 The control-flow of delimited continuations

When an effect is performed by one *Player* and gets propagated beyond its immediate controlled scope, it initiates a sequence of effectful moves until the effect is handled or reaches *top-level*.

To visualise this part of the interaction, we will zoom in at the computation underlying it in the concrete example of 3.4.

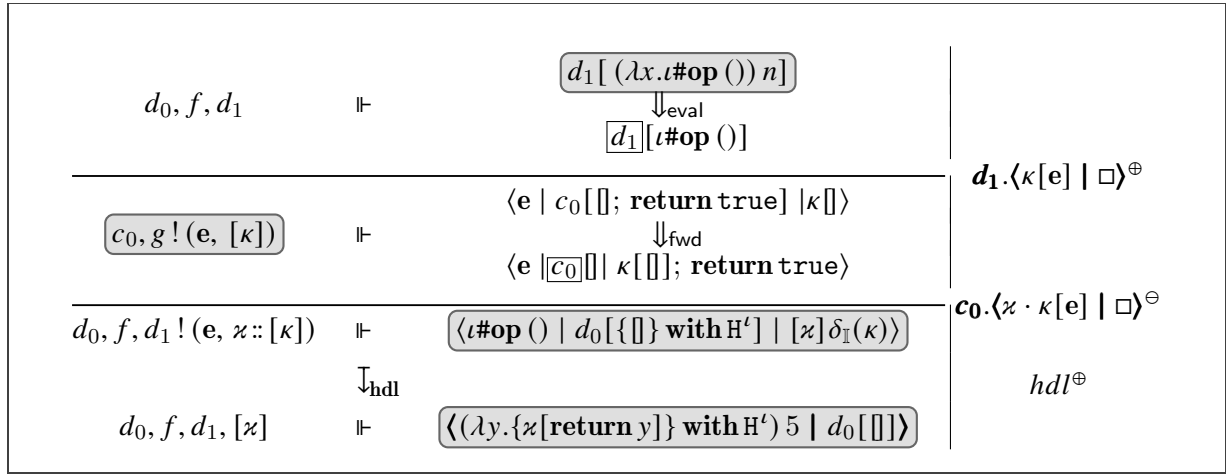


Fig. 13. effectful forward trace.

In figure.13, we only highlight the player responsible for triggering the effect (in its active and passive states). The resulting effectful sequence is $d_1. \langle \kappa_g[e] \mid \Box \rangle^\oplus c_0. \langle \kappa_d :: \kappa_g[e] \mid \Box \rangle^\ominus$, which can be understood as a *pseudo-copycat* sequence in which *Opponent* forwards the computation $\langle e \mid d_1[\Box] \circ \kappa_g[\Box] \rangle$ as is to the abstract continuation $c_0[\Box]$.

Now if we wish to generalize this sequence to one of arbitrary length n , $\mathbb{I} \xRightarrow{\text{fwd}} \mathbb{J}$, the generated configuration will always be of the following shape:

$$\mathbb{I} \xRightarrow{\text{fwd}} \mathbb{J} \quad \mathbb{J} \xrightarrow{\text{hdl}}_{\text{fwd}} \overline{\langle \langle \iota \# \text{op } v \mid T[\{\Box\} \text{ with } H] \circ S_{n+1}[\kappa_n[S_n[\cdots \kappa_0[S_0[\cdots]]]] \mid \Sigma] \rangle \rangle}$$

Consequently, the captured continuation will be of the shape:

$$\lambda x. \{ S_{n+1}[\kappa_n[S_n[\cdots \kappa_0[S_0[\text{return } x]]] \cdots]] \} \text{ with } H$$

In the case the continuation is discarded and not used, the control-flow stack remains intact. Otherwise, every call to the captured delimited continuation entails exactly one query addressed to each one of the n environment controlled fragments; *i.e.* a call to each of the abstract stack frames κ_i in the order in which they have been disclosed.

3.4.2 Well-bracketing LTS

We define the well-bracketing LTS as $\mathcal{L}_{wb} := (\mathcal{W}, \mathcal{M}, \rightarrow_{wb})$ where the configurations are described below and the transitions $\xrightarrow{m}_{wb}$ are defined in figure 16.

$$\begin{aligned} \text{effect forwarding info. } \mathcal{F} \times \mathcal{E} \ni \eta & ::= \emptyset & (\text{absence of effect}) \\ & \mid (f, e) & (\text{effect } e \text{ forwarded through } f) \\ \text{configurations } \mathcal{W} \ni \mathbb{W}, \mathbb{U} & := \langle \pi \mid \eta \mid \phi \rangle \end{aligned}$$

where $\pi \in \text{List}(C \cup \mathcal{K})$, $f \in \mathcal{F} = \text{List}(\mathcal{K})$ denote a *control-flow stack* and a *call frame*, respectively, and $\phi \in \wp(\mathcal{F})$ denotes the set of all captured call frames.

$$\begin{aligned} \text{forward}(\emptyset, \langle e \mid c \circ \kappa \rangle^\ominus) & := ([], e) \\ \text{forward}(\emptyset, \langle e \mid c[\kappa] \circ \kappa \rangle^\ominus) & := ([], e) \\ \text{forward}(\eta, o) & := \eta \\ \\ \text{call/ret}(c :: \pi, _, \langle \underline{A} \mid c \circ _ \rangle^\ominus) & := \pi \\ \text{call/ret}(\kappa :: \pi, _, \langle \underline{A} \mid d[\kappa] \circ _ \rangle^\ominus) & := \pi \\ \text{call/ret}(\pi, \phi, \langle \underline{A} \mid d[\kappa] \circ _ \rangle^\ominus) & := f \# \pi \\ & (\text{when } \kappa :: f \in \phi) \\ \\ \frac{\eta' = \text{forward}(\eta, o) \quad \pi' = \text{call/ret}(\pi, \phi, o)}{\langle \pi \mid \eta \mid \phi \rangle \xrightarrow{o}_{wb} \langle \pi' \mid \eta' \mid \phi \rangle} \\ \\ \frac{}{\langle \pi \mid (f, e) \mid \phi \rangle \xrightarrow{hdl^\ominus}_{wb} \langle \pi \mid \emptyset \mid \phi \cup \{f\} \rangle} \end{aligned}$$

Fig. 14. Opponent transitions

$$\begin{aligned} \text{forward}((f, e), \langle e \mid c \circ \kappa :: r \rangle^\oplus) & := (f \# [\kappa], e) \\ \text{forward}((f, e), \langle e \mid c[\kappa] \circ \kappa :: r \rangle^\oplus) & := (f \# [\kappa], e) \\ \text{forward}(\emptyset, p) & := \emptyset \\ \\ \text{call/ret}(\pi, _, \langle \underline{A} \mid c[\kappa] \circ _ \rangle^\oplus) & := c :: \pi \\ \text{call/ret}(\pi, _, \langle f \mid A \mid c \rangle^\oplus) & := c :: \pi \\ \text{call/ret}(\pi, _, p) & := \pi \\ \\ \frac{\pi' = \text{call/ret}(\pi, p) \quad \eta' = \text{forward}(\eta, p)}{\langle \pi \mid \eta \mid \phi \rangle \xrightarrow{p}_{wb} \langle \pi' \mid \eta' \mid \phi \rangle} \\ \\ \frac{}{\langle \pi \mid \emptyset \mid \phi \rangle \xrightarrow{hdl^\oplus}_{wb} \langle \pi \mid \emptyset \mid \phi \rangle} \end{aligned}$$

Fig. 15. Proponent transitions

Fig. 16. $\mathcal{L}_{wb}$ transitions.

3.5 The OGS LTS

The OGS LTS $(\mathcal{O}, \mathcal{M}, \rightarrow_{ogs})$ is the transition system that can generate the interaction traces² compatible with what our language $(\Lambda_{\text{eff}}, \mapsto_{\text{op}})$ permits.

It is defined as the product of $\mathcal{L}_{AI}$, $\mathcal{L}_{\mathcal{T}}$ and $\mathcal{L}_{wb}$ where the underlying labelled transitions systems are synchronised on $\mathcal{M} \cup \{hdl^\oplus, hdl^\ominus\}$. The configurations in $\mathcal{O}$ satisfy some conditions that guarantee the well-typedness of $\mathcal{L}_{AI}$; in that *Opponent* and *Proponent* agree on types during their interaction, in addition to ensuring that *Opponent*'s use of continuations is well-bracketed.

In the following we will denote $\langle \mathbb{I} \parallel \mathbb{T} \parallel \mathbb{W} \rangle \in \mathcal{A} \times \mathcal{T} \times \mathcal{W}$ any triple of configurations satisfying $\mathbb{I} \sharp \mathbb{T}$ and **Compat**($\mathbb{I}, \mathbb{W}$), where the second proposition morally means that the control-flow history in $\mathbb{W}$ is *compatible* with the interactive information in $\mathbb{I}$. The formal definition of this predicate will be later introduced in section E.2.

² For a reader familiar with *game semantics*, this LTS specifies the *plays*.

3.5.1 Embedding the interactive language

Finally, we define an interactive term embedding into an initial *active* $\mathcal{L}_{\text{OGS}}$ configuration and environment embedding into an initial *passive* $\mathcal{L}_{\text{OGS}}$ configuration.

Definition 3.6 (term embedding) Given an interactive term $\Sigma; \Gamma \Vdash_c M$, we define its embedding

$$\langle \Sigma; \Gamma \Vdash_c M \rangle_{\text{ogs}}^{\oplus} := \langle \langle (M \mid \Sigma); \varepsilon \rangle \parallel \langle \Gamma; \emptyset \mid c_f : \neg \mathbb{1}; \emptyset \rangle \parallel \langle [d]; \emptyset; \emptyset \rangle \rangle$$

with d the unique continuation name of Γ .

Definition 3.7 (environment embedding) Given a name assignment $\Sigma; c_f : \neg \mathbb{1} \vdash \gamma : \Gamma$, we define its embedding

$$\langle \Sigma; c_f : \neg \mathbb{1} \vdash \gamma : \Gamma \rangle_{\text{ogs}}^{\ominus} := \langle \langle \emptyset; \gamma \rangle \parallel \langle c_f : \neg \mathbb{1}; \emptyset \mid \Gamma; \emptyset \rangle \parallel \langle [c_f]; \emptyset; \emptyset \rangle \rangle$$

4 The OGS model

4.1 Interpretation of expressions

Following our equivalence of programs introduced in definition 2.3, we only observe coverage, *i.e.* a full evaluation into a value returner of the shape **return** v . We introduce a notion of *complete trace* in order to capture this *complete* evaluation.

Definition 4.1 (complete traces): A trace $\mathbf{t} \mathbf{m} \in \mathbf{Tr}_{\text{ogs}}$ is said to be *complete* and written $\mathcal{CP}(\mathbf{t} \mathbf{m})$ when:

$$\mathcal{CP}(\mathbf{t} \mathbf{m}) := \begin{cases} \langle \square \mid \emptyset \mid \emptyset \rangle \xrightarrow[\text{wb}]{\mathbf{t} \mathbf{m}} \langle \square \mid \eta \mid \phi \rangle \text{ for some } \eta \text{ and } \phi \\ \mathbf{m} = \mathbf{c}. \langle \text{ret } A \mid \square \rangle^{\ominus} \text{ for some } A \text{ and } c. \end{cases}$$

Accordingly, we define the set of complete *traces* of a configuration:

$$\mathbf{CTr}_{\text{ogs}}(\mathbb{G}) := \{ \mathbf{t} \in \mathbf{Tr}_{\text{ogs}}(\mathbb{G}) \mid \mathcal{CP}(\mathbf{t}) \}$$

4.2 Soundness of the model

Theorem 4.2 (Adequacy) Given an interactive term $\Sigma; \Gamma \Vdash_c M$ and a compatible name assignment $\Sigma'; c_f : \neg \mathbb{1} \vdash \gamma : \Gamma$, then by writing $\mathbb{P}$ and $\mathbb{E}$ for their respective *ogs* embeddings, we have:

$$\begin{aligned} \exists \Sigma'' \supseteq \Sigma' \cdot \Sigma. (M\{\gamma\} \mid \Sigma' \cdot \Sigma) \Downarrow_{\text{op}} ([c_f] \text{ return } () \mid \Sigma'') \\ \iff \\ \exists \mathbf{t} \in \mathbf{CTr}_{\text{ogs}}(\mathbb{P}). \mathbf{t}^{\perp} \mathbf{c}_f. \langle \text{ret } () \mid \square \rangle^{\oplus} \in \mathbf{CTr}_{\text{ogs}}(\mathbb{E}) \end{aligned}$$

The proof of the previous adequacy theorem 4.2 is detailed in section C. It relies on the introduction of another LTS (*cf.* section B) that formalizes the notion of *concrete interaction* of example 3.4. This allows to define semantic notions of composition and observation (at the level of configurations and complete traces) which are then shown, in sections D and E, to coincide with their syntactic/operational counterparts.

Theorem 4.3 (Soundness) Taking M_1, M_2 two interactive terms such that both $\Sigma; \Gamma \Vdash_c M_i$ (for $i \in \{1, 2\}$), suppose that $\mathbf{CTr}(\langle \Sigma; \Gamma \vdash M_1 \rangle) \subseteq \mathbf{CTr}(\langle \Sigma; \Gamma \vdash M_2 \rangle)$. Then $\Sigma; \Gamma \vdash M_1 \preceq_{\text{ciu}} M_2$.

Proof. We take a continuation name c_f , an instance context $\Sigma' \supseteq \Sigma$ and a name assignment γ such that $\Sigma'; c_f : \neg \mathbb{1} \vdash \gamma : \Gamma$. We write $\mathbb{G}_{P,i}$ for $\langle \Sigma; \Gamma \Vdash_c T_i \rangle_{\text{ogs}}^{\oplus}$, and $\mathbb{G}_O$ for $\langle \Sigma'; c_f : \neg \mathbb{1} \vdash \gamma : \Gamma \rangle_{\text{ogs}}^{\ominus}$.

Suppose that $(M_1\{\gamma\} \mid \Sigma') \Downarrow_{\text{op}} [c_f]()$. Then from Theorem 4.2, we get the existence of a complete trace $\mathbf{t} \in \mathbf{CTr}(\mathbb{G}_{P,1})$ such that $\mathbf{t}_1^{\perp} \mathbf{c}_f. \langle \text{ret } () \mid \square \rangle^{\oplus} \in \mathbf{CTr}(\mathbb{G}_O)$. From $\mathbf{CTr}(\mathbb{G}_{P,1}) \subseteq \mathbf{CTr}(\mathbb{G}_{P,2})$, we get that $\mathbf{t} \in \mathbf{CTr}(\mathbb{G}_{P,2})$. Finally, from Theorem 4.2, we get that $(M_2\{\gamma\} \mid \Sigma') \Downarrow_{\text{op}} [c_f]()$, as we wanted. $\square$

4.3 Counterexample to full-abstraction

Recall how the interaction of the term $u := \text{let } y = \mathbf{new} \ \mathbb{E} \text{ in } \{f(\lambda x.y\#\mathbf{op}())\} \mathbf{with} \ H^y$ (from example 3.4) with the environment given by $C := \text{let } f = (\lambda g.g \ \mathbf{v}; \mathbf{return} \ \mathbf{true}) \text{ in } []$ was represented by the following trace

$$s_u = f.\langle \Box g \mid d \rangle^\oplus g.\langle \Box A \mid c \rangle^\ominus c.\langle \kappa[e] \mid \Box \rangle^\oplus d.\langle \kappa_c :: \kappa[e] \mid \Box \rangle^\ominus \kappa_c.\langle \Box[\mathbf{ret} \ 5] \mid d' \rangle^\oplus$$

Now considering the term $t := f(\lambda x.5)$, that is contextually equivalent to u , and observing how it interacts with the same environment C , we get:

$$\begin{aligned} \langle c_f[t_1], \varepsilon, \emptyset \rangle &\xrightarrow{f.\langle \Box g \mid d \rangle^\oplus}_1 \langle [g \mapsto (\lambda x.5); d \mapsto c_f[[]]; \emptyset \rangle \\ &\xrightarrow{g.\langle \Box A \mid c \rangle^\ominus}_1 \langle c[(\lambda x.5)] \ A; [g \mapsto (\lambda x.5); d \mapsto c_f[[]]; \emptyset \rangle \\ &\xrightarrow{c.\langle \mathbf{ret} \ 5 \mid \Box \rangle^\oplus}_1 \langle [g \mapsto (\lambda x.5); d \mapsto c_f[[]]; \emptyset \rangle \\ &\xrightarrow{d.\langle \mathbf{ret} \ \mathbf{true} \mid \Box \rangle^\ominus}_1 \langle c_f[\mathbf{return} \ \mathbf{true}], [g \mapsto (\lambda x.5); d \mapsto c_f[[]]; \emptyset \rangle \\ &\xrightarrow{c_f.\langle \mathbf{ret} \ \mathbf{true} \mid \Box \rangle^\oplus}_1 \langle [g \mapsto (\lambda x.5); d \mapsto c_f[[]]; \emptyset \rangle \end{aligned}$$

that is an interaction witnessed by the trace s_t

$$s_t = f.\langle \Box g \mid d \rangle^\oplus g.\langle \Box A \mid c \rangle^\ominus c.\langle \mathbf{ret} \ 5 \mid \Box \rangle^\oplus d.\langle \mathbf{ret} \ \mathbf{true} \mid \Box \rangle^\ominus c_f.\langle \mathbf{ret} \ \mathbf{true} \mid \Box \rangle^\oplus$$

It is clear that s_u is not a trace generated by t , and s_t is not a trace generated by u , so that their respective set of complete traces are incomparable. Therefore, a complete OGS model requires a notion of equivalence coarser than equality of traces, that would identify s_t and s_u .

5 Conclusion and related work

We have provided a sound interactive model for a language with dynamically-scoped effect handlers and fresh generation of effect instances. Our model does not allow so far exchange of effect instances between the program and its environment, that would necessitate to keep track of such disclosed instances in a specific set, to enforce a non-omniscience property as in [12]. The well-bracketing LTS would also have to keep track of the exchanged delimited continuations when the program performs a public effect (using a disclosed instance that is) because the environment, in this case, would be able to handle it, and thus its behaviour must be bound accordingly.

In future work, we will present a notion of equivalence between set of complete traces, coarser than equality, that we conjecture to be fully-abstract.

Game semantics has a long history of providing a fully-abstract interactive models for languages with control operators, starting from the relaxation of the well-bracketed condition by Laird to capture control operators like call/cc [18] and exceptions [19]. Game models for both statically bound, first-class continuations and locally declared, dynamically bound prompts were presented in [21], via a monadic presentation of such effects. In this work, prompts and exceptions cannot be referred by their names and passed around.

To represent them as values, nominal game semantics has been developed as a versatile framework to handle generative effects like dynamic name creation [1], higher-order references [26] and generative exceptions and handlers [27].

Operational techniques like applicative, normal-form and environmental bisimulations has also been developed for higher-order language with fine-grained control operators like static delimited continuations via shift and reset operations [6], dynamic ones via prompt and reset [4], and algebraic effects and handlers were considered in [7]. This last work is the most relevant to us, however they do not consider fresh generation of instances. Using a notion of Kripke normal-form bisimulations as introduced in [17,14], that is directly derived from an OGS model, it would be interesting to explore the development of normal form bisimulations for such fresh generation of effect instances.

References

- [1] Abramsky, S., D. R. Ghica, A. S. Murawski, C. L. Ong and I. D. B. Stark, *Nominal games and full abstraction for the nu-calculus*, in: *19th IEEE Symposium on Logic in Computer Science (LICS 2004), 14-17 July 2004, Turku, Finland, Proceedings*, pages 150–159, IEEE Computer Society (2004).
<https://doi.org/10.1109/LICS.2004.1319609>
- [2] Abramsky, S., R. Jagadeesan and P. Malacaria, *Full abstraction for pcf*, *Inf. Comput.* **163**, page 409–470 (2000), ISSN 0890-5401.
<https://doi.org/10.1006/inco.2000.2930>
- [3] Accattoli, B., P. Barenbaum and D. Mazza, *Distilling abstract machines*, in: *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming, ICFP '14*, pages 363–376, Association for Computing Machinery, New York, NY, USA (2014), ISBN 978-1-4503-2873-9.
<https://doi.org/10.1145/2628136.2628154>
- [4] Aristizábal, A., D. Biernacki, S. Lenglet and P. Polesiuk, *Environmental bisimulations for delimited-control operators with dynamic prompt generation*, *Log. Methods Comput. Sci.* **13** (2017).
[https://doi.org/10.23638/LMCS-13\(3:27\)2017](https://doi.org/10.23638/LMCS-13(3:27)2017)
- [5] Bauer, A. and M. Pretnar, *Programming with algebraic effects and handlers*, *Journal of Logical and Algebraic Methods in Programming* **84**, pages 108–123 (2015), ISSN 2352-2208. Special Issue: The 23rd Nordic Workshop on Programming Theory (NWPT 2011) Special Issue: Domains X, International workshop on Domain Theory and applications, Swansea, 5-7 September, 2011.
<https://doi.org/https://doi.org/10.1016/j.jlamp.2014.02.001>
- [6] Biernacki, D., S. Lenglet and P. Polesiuk, *Bisimulations for delimited-control operators*, *Log. Methods Comput. Sci.* **15** (2019).
[https://doi.org/10.23638/LMCS-15\(2:18\)2019](https://doi.org/10.23638/LMCS-15(2:18)2019)
- [7] Biernacki, D., S. Lenglet and P. Polesiuk, *A complete normal-form bisimilarity for algebraic effects and handlers*, in: *Formal Structures for Computation and Deduction* (2020).
- [8] Biernacki, D., M. Piróg, P. Polesiuk and F. Sieczkowski, *Binders by day, labels by night: effect instances via lexically scoped handlers*, *Proceedings of the ACM on Programming Languages* **4**, pages 1–29 (2019).
- [9] Brachthäuser, J. I., P. Schuster and K. Ostermann, *Effects as capabilities: effect handlers and lightweight effect polymorphism*, *Proc. ACM Program. Lang.* **4**, pages 126:1–126:30 (2020).
<https://doi.org/10.1145/3428194>
- [10] Danos, V., H. Herbelin and L. Regnier, *Game semantics and abstract machines*, in: *Proceedings 11th Annual IEEE Symposium on Logic in Computer Science*, pages 394–405, IEEE (1996).
- [11] de Vilhena, P. E. and F. Pottier, *A type system for effect handlers and dynamic labels*, in: *Programming Languages and Systems: 32nd European Symposium on Programming, ESOP 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2023, Paris, France, April 22–27, 2023, Proceedings*, pages 225–252, Springer Nature Switzerland Cham (2023).
- [12] Ghica, D. R. and N. Tzevelekos, *A system-level game semantics*, *Electronic Notes in Theoretical Computer Science* **286**, pages 191–211 (2012).
- [13] Hillerström, D., S. Lindley and R. Atkey, *Effect handlers via generalised continuations* **30**, page e5, ISSN 0956-7968, 1469-7653.
<https://doi.org/10.1017/S0956796820000040>
- [14] Hirschhoff, D., G. Jaber and E. Prebet, *Deciding contextual equivalence of v-calculus with effectful contexts*, in: O. Kupferman and P. Sobocinski, editors, *Foundations of Software Science and Computation Structures - 26th International Conference, FoSSaCS 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2023, Paris, France, April 22–27, 2023, Proceedings*, volume 13992 of *Lecture Notes in Computer Science*, pages 24–45, Springer (2023).
https://doi.org/10.1007/978-3-031-30829-1_2
- [15] Hyland, J. and C.-H. Ong, *On full abstraction for pcf*, *Inf. Comput.* **163**, page 285–408 (2000), ISSN 0890-5401.
<https://doi.org/10.1006/inco.2000.2917>
- [16] Jaber, G. and A. S. Murawski, *Complete trace models of state and control*, *Programming Languages and Systems* **12648**, page 348 (2021).

- [17] Jaber, G. and A. S. Murawski, *Compositional relational reasoning via operational game semantics*, in: *36th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2021, Rome, Italy, June 29 - July 2, 2021*, pages 1–13, IEEE (2021).
<https://doi.org/10.1109/LICS52264.2021.9470524>
- [18] Laird, J., *Full abstraction for functional languages with control*, in: *Proceedings of Twelfth Annual IEEE Symposium on Logic in Computer Science*, pages 58–67 (1997).
<https://doi.org/10.1109/LICS.1997.614931>
- [19] Laird, J., *A fully abstract game semantics of local exceptions*, in: *16th Annual IEEE Symposium on Logic in Computer Science, Boston, Massachusetts, USA, June 16-19, 2001, Proceedings*, pages 105–114, IEEE Computer Society (2001).
<https://doi.org/10.1109/LICS.2001.932487>
- [20] Laird, J., *A fully abstract trace semantics for general references*, in: *Proceedings of the 34th International Conference on Automata, Languages and Programming, ICALP'07*, page 667–679, Springer-Verlag, Berlin, Heidelberg (2007), ISBN 3540734198.
- [21] Laird, J., *Combining control effects and their models: Game semantics for a hierarchy of static, dynamic and delimited control effects*, *Ann. Pure Appl. Log.* **168**, pages 470–500 (2017).
<https://doi.org/10.1016/J.APAL.2016.10.011>
- [22] Lassen, S. B. and P. B. Levy, *Typed normal form bisimulation for parametric polymorphism*, in: *2008 23rd Annual IEEE Symposium on Logic in Computer Science*, pages 341–352, IEEE (2008).
- [23] Levy, P. B., *Call-By-Push-Value: A Functional/Imperative Synthesis*, volume 2 of *Semantics Structures in Computation*, Springer (2004), ISBN 1-4020-1730-8.
- [24] Mason, I. and C. Talcott, *Equivalence in functional languages with effects*, *Journal of Functional Programming* **1**, page 287–327 (1991).
<https://doi.org/10.1017/S0956796800000125>
- [25] Morris Jr, J. H., *Lambda-calculus models of programming languages.*, Ph.D. thesis, Massachusetts Institute of Technology (1969).
- [26] Murawski, A. S. and N. Tzevelekos, *Game semantics for good general references*, in: *2011 IEEE 26th Annual Symposium on Logic in Computer Science*, pages 75–84, IEEE (2011).
- [27] Murawski, A. S. and N. Tzevelekos, *Game semantics for nominal exceptions*, in: *Foundations of Software Science and Computation Structures: 17th International Conference, FOSSACS 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014, Proceedings 17*, pages 164–179, Springer (2014).
- [28] Plotkin, G. and J. Power, *Semantics for algebraic operations*, *Electronic Notes in Theoretical Computer Science* **45**, pages 332–345 (2001).
- [29] Plotkin, G. and J. Power, *Computational effects and operations: An overview* (2002).
- [30] Plotkin, G. D. and M. Pretnar, *Handling algebraic effects*, *Logical methods in computer science* **9** (2013).
- [31] Zhang, Y. and A. C. Myers, *Abstraction-safe effect handlers via tunneling*, *Proc. ACM Program. Lang.* **3**, pages 5:1–5:29 (2019).
<https://doi.org/10.1145/3290318>

A Surface Programming Language Λ_{eff}

A.1 Operational semantics

$$\begin{aligned}
((\lambda x. \mathbf{t}) \mathbf{v} \mid \Sigma) &\rightarrow_{\text{op}} (\mathbf{t}\{x := \mathbf{v}\} \mid \Sigma) \\
(\text{let } x = \mathbf{return} \mathbf{v} \text{ in } \mathbf{t} \mid \Sigma) &\rightarrow_{\text{op}} (\mathbf{t}\{x := \mathbf{v}\} \mid \Sigma) \\
(\mathbf{new} \mathbb{E} \mid \Sigma) &\rightarrow_{\text{op}} (\mathbf{return} \iota \mid \Sigma \cdot [\iota \mapsto \mathbb{E}]) \\
(\{\mathbf{return} \mathbf{v}\} \mathbf{with} \mathbf{H} \mid \Sigma) &\rightarrow_{\text{op}} (\mathbf{t}\{x := \mathbf{v}\} \mid \Sigma) \quad \text{where } \mathbf{H}^{\text{ret}} = \{\mathbf{return} \ x \mapsto \mathbf{t}\} \\
(\{S[\iota\#\mathbf{op} \ \mathbf{v}]\} \mathbf{with} \mathbf{H} \mid \Sigma) &\rightarrow_{\text{op}} (\mathbf{t}\{x := \mathbf{v}\} \{\kappa := \lambda y. \{S[\mathbf{return} \ y]\} \mathbf{with} \mathbf{H}\} \mid \Sigma) \\
&\quad \text{where } \mathbf{H}^{\text{op}} = \{\iota\#\mathbf{op} \ x \ \kappa \mapsto \mathbf{t}\} \text{ and } \iota\#\mathbf{op} \notin \mathbf{hdl}(\mathbf{S})
\end{aligned}$$

$$\frac{(\mathbf{t} \mid \Sigma) \rightarrow_{\text{op}} (\mathbf{u} \mid \Sigma')}{(S[\mathbf{t}] \mid \Sigma) \rightarrow_{\text{op}} (S[\mathbf{u}] \mid \Sigma')}$$

Fig. A.1. standard operational semantics.

A.2 Operational semantics with explicit propagation

$$\begin{aligned}
\langle (\lambda x. \mathbf{t}) \mathbf{v} \mid S \mid \Sigma \rangle &\mapsto_{\text{eval}} \langle \mathbf{t}\{x := \mathbf{v}\} \mid S \mid \Sigma \rangle \\
\langle \mathbf{new} \mathbb{E} \mid S \mid \Sigma \rangle &\mapsto_{\text{eval}} \langle \mathbf{return} \iota \mid S \mid \Sigma \cdot [\iota \mapsto \mathbb{E}] \rangle \\
\langle \mathbf{return} \mathbf{v} \mid S[\text{let } x = [] \text{ in } \mathbf{t}] \mid \Sigma \rangle &\mapsto_{\text{eval}} \langle \mathbf{t}\{x := \mathbf{v}\} \mid S \mid \Sigma \rangle \\
\langle \mathbf{return} \mathbf{v} \mid S[\{\} \mathbf{with} \mathbf{H}] \mid \Sigma \rangle &\mapsto_{\text{eval}} \langle \mathbf{t}\{x := \mathbf{v}\} \mid S \mid \Sigma \rangle \\
&\quad \text{where } \mathbf{H}^{\text{ret}} = \{\mathbf{return} \ x \mapsto \mathbf{t}\} \\
\langle \iota\#\mathbf{op} \ \mathbf{v} \mid S[\{\} \mathbf{with} \mathbf{H}] \circ K \mid \Sigma \rangle &\mapsto_{\text{eval}} \langle \mathbf{t}\{x := \mathbf{v}\} \{\kappa := \lambda y. \{K[\mathbf{return} \ y]\} \mathbf{with} \mathbf{H}\} \mid S \mid \Sigma \rangle \\
&\quad \text{where } \mathbf{H}^{\text{op}} = \{\iota\#\mathbf{op} \ x \ \kappa \mapsto \mathbf{t}\}
\end{aligned}$$

$$\begin{aligned}
\langle \iota\#\mathbf{op} \ \mathbf{v} \mid S[\mathbf{E}] \circ K \mid \Sigma \rangle &\mapsto_{\text{fwd}} \langle \iota\#\mathbf{op} \ \mathbf{v} \mid S \circ \mathbf{E}[K] \mid \Sigma \rangle \quad \text{when } \iota\#\mathbf{op} \notin \mathbf{hdl}(\mathbf{E}) \\
\langle \mathbf{E}[\mathbf{t}] \mid S \mid \Sigma \rangle &\mapsto_{\text{push}} \langle \mathbf{t} \mid S[\mathbf{E}] \mid \Sigma \rangle
\end{aligned}$$

(a) reduction rules of the abstract machine.

$$\begin{aligned}
\mapsto_{\text{op}} &:= \mapsto_{\text{struct}}^* \mapsto_{\text{eval}} \mapsto_{\text{struct}}^* \\
&\quad \frac{(\mathbf{t} \mid \Sigma) \rightarrow_{\text{op}} (S[K[\mathbf{u}]] \mid \Sigma')}{\langle \mathbf{t} \mid [] \mid \Sigma \rangle \mapsto_{\text{op}} \langle \mathbf{u} \mid S \circ K \mid \Sigma' \rangle}
\end{aligned}$$

(b) simulating the relation $\rightarrow_{\text{op}}$.

Fig. A.2. finer operational semantics

B Concrete Interaction

$\mathcal{L}_{ci}$ can be seen as an abstract machine that performs a variant of the linear head reduction, which is known to correspond to interaction in game semantics [10], (having a global environment γ , it is similar to the Milner Abstract Machine [3]), computing the interaction between a program configuration and an environment configuration that are *composable*.

The purpose of the concrete interaction is to *evaluate* a term inside an environment while exhibiting the *interaction* between the two that gets obscured by plain *syntax substitution*. To this effect, the program being evaluated and its environment will remain decoupled, but will each carry an *information* component that will be threaded through their interaction. This component keeps track of the history of the interaction and it consists of a map between the names and the *concrete* code of codata that each party has disclosed and rendered accessible to the other.

B.1 Concrete interaction LTS

The concrete interaction describes how a term and a complementary *concrete* environment interact by disclosing information and alternate between passive and active states. It emphasizes the interaction points between the two by highlighting the information they disclose to each other. At every stage of the interaction, the passive player is described by an accumulative component tracking its past contribution to the interaction, whereas the active player description contains in addition an interactive term representing the current *active* computation.

It is a polarized abstract machine whose configurations involve a *passive* and a *decoupled* complementary *passive* $\mathcal{L}_{AI}$ configurations (cf. 3.2).

We formalize this notion of complementarity by the following definition.

Definition B.1 (Complementarity) Let $\mathbb{P} = \langle \Sigma_a; \gamma_a; \xi_a \rangle$, $\mathbb{A} = \langle (M \mid \Sigma_p); \gamma_p; \delta_p \rangle \in \mathcal{A}$ such that $M = \langle \tau \mid S \circ T \rangle$ and $c_f \in C$. We say that $\mathbb{P}$ and $\mathbb{A}$ are c_f -complementary and write $\mathbb{P} \dashv_c \mathbb{A}$ when:

- $\Sigma_a \cap \Sigma_p = \emptyset$ and $\text{dom}(\gamma_a) \cup \text{dom}(\gamma_p) = \emptyset$
- $\exists y \in \{a, p\}. \exists d \in \text{dom}(\gamma_y). c_f \in \text{supp}(\gamma_y(d))$
- $\forall y \in \{a, p\}. \forall d \in \text{dom}(\gamma_y). (\text{supp}(\gamma_y(d)) \setminus \{c_f\}) \subseteq \text{dom}(\gamma_{y^\perp})$
- $\text{supp}(\text{codom}(\xi_a)) \subseteq \text{dom}(\gamma_p)$ and $\text{supp}(\text{codom}(\delta_p)) \subseteq \text{dom}(\gamma_a)$
- $\text{supp}(\tau), \text{supp}(S) \subseteq \text{dom}(\gamma_a)$ and $\text{supp}(T) \subseteq \text{dom}(\gamma_a) \cup \text{dom}(\xi_a)$.
- There exists a type configuration $\mathbb{T} = \langle \Gamma_o; \Delta_o \mid \Gamma_p; \Delta_p \rangle \in \mathcal{T}$ and a type τ s.t. either:
 $\mathbb{P} \vDash \langle \Gamma_o, c_f : \neg\tau; \Delta_o \mid \Gamma_p; \Delta_p \rangle$ and $\mathbb{A} \vDash \mathbb{T}^\perp$ or $\mathbb{P} \vDash \mathbb{T}$ and $\mathbb{A} \vDash \langle \Gamma_p, c_f : \neg\tau; \Delta_p \mid \Gamma_o; \Delta_o \rangle$.

We will sometimes omit the index c on the relation $\dashv_c$ in contexts that do not require us to be explicit and just write $\mathbb{P} \dashv \mathbb{A}$.

In the following, we will use the notation $\langle \mathbb{P} \dashv \mathbb{A} \rangle$ to denote the couple $(\mathbb{P}, \mathbb{A}) \in \mathcal{A}^2$ satisfying $\mathbb{P} \dashv \mathbb{A}$.

B.1.1 configurations and transitions

$$\text{configurations} \quad \text{Confs}_{ci} \ni \mathbb{C}, \mathbb{D} ::= \langle \mathbb{P} \dashv \mathbb{A} \rangle$$

The CI transitions are given by $\mapsto_{ci} := \overset{\text{eval}}{\mapsto}_{ci} \cup \overset{\text{dual}}{\mapsto}_{ci} \cup \overset{\text{hdl}}{\mapsto}_{ci}$ and are defined in fig. B.1. Similar to $\mathcal{L}_{AI}$, $\overset{\text{eval}}{\mapsto}_{ci}$ corresponds to evaluating a term through $\mapsto_{\text{eval}}$ down to an normal form, while $\overset{\text{hdl}}{\mapsto}_{ci}$ marks the end of effect propagation whereby the *active* player gets to capture the fragments of the *forward stack* that belong to the environment.

On the other hand, $\overset{\text{dual}}{\mapsto}_{ci}$ acts on configurations in normal form by dualizing the perspective of the computation; i.e taking the perspective of the environment.

The dualizing process

It is defined as the composition of two processes **abstract** and **concretize**; The polarity of the *abstract normal form* $a.p$ produced by **abstract** is switched in order to take the environment's perspective, then

$$\begin{array}{c}
\text{EVALUATING} \frac{(M \mid \Sigma_p \cdot \Sigma_a) \mapsto_{\text{op}} (N \mid \Sigma_p \cdot \Sigma'_a)}{\langle \Sigma_p; \gamma_p \Vdash (M \mid \Sigma_a); \gamma_a \rangle \xrightarrow{\text{eval}}_{\text{ci}} \langle \Sigma_p; \gamma_p \Vdash (N \mid \Sigma'_a); \gamma_a \rangle} \\
\\
\text{HANDLING} \langle \Sigma_a; \gamma_p; ?(\rightarrow, \rightarrow, \delta_p) \Vdash (M \mid \Sigma_p); \gamma_a; \delta_a \rangle \xrightarrow{\text{hdl}}_{\text{ci}} \langle \Sigma_a; \gamma_p \cdot \delta_p \Vdash (M \mid \Sigma_p); \gamma_a \rangle \\
\text{(a) Internal evaluation} \\
\\
\text{DUALIZING} \frac{\text{abstract}(N_f, \delta_p) \not\approx (\alpha.p, \gamma_{\text{abs}}, \xi_{\text{abs}}) \quad \text{concretize}(\alpha.p, \gamma_p, \xi_p) = M}{\langle \Sigma_a; \gamma_p; \xi_p \Vdash (N_f \mid \Sigma_p); \gamma_a; \delta_a \rangle \xrightarrow{\text{dual}}_{\text{ci}} \langle \Sigma_p; \gamma_a \cdot \gamma_{\text{abs}}; \xi_a[\xi_{\text{abs}}] \Vdash (M \mid \Sigma_a); \gamma_p; \xi_p \rangle} \\
\text{(b) Interaction interface}
\end{array}$$

Fig. B.1. transitions of the Concrete Interaction LTS

the concretization is carried out on $\alpha.p$ as explained above.

Abstracting a *co-pattern* into the shapes $\langle r[e] \mid S[\square] \rangle$ or $\langle r[e] \mid \square \rangle$; that is an abstract effect e with its delimiting abstract evaluation stack r is a gradual process. The fragments κ making up r cannot be computed from the syntax alone, an interaction has to force the environment to perform the necessary "abstract" $\mapsto_{\text{fwd}}$ reductions on stuck terms such as $\langle \iota \# \text{op } v \mid S[\kappa] \rangle \circ T$ or $\langle e \mid c \mid \circ T \rangle$ in order to progressively compute r . The exchanged delimited continuations are gradually put in δ , whose disclosure and concretization is delayed until the effect has been effectively handled (cf. handling rule in Fig. B.1).

B.2 Properties of the CI LTS

B.2.1 Correctness

Lemma B.2 (Complementarity preservation) *Let $P \in \text{Confs}_{\text{ci}}^{\ominus}$, $A \in \text{Confs}_{\text{ci}}^{\oplus}$ such that $P \Vdash A$. If $\langle P \Vdash A \rangle \mapsto_{\text{ci}} \langle P' \Vdash A' \rangle$, then $P' \Vdash A'$.*

Proof. Let's write $A = \langle (M \mid \Sigma_a); \gamma_p; \delta_p \rangle$, $P = \langle \Sigma_p; \gamma_a; \xi_a \rangle$ and $M = \langle t \mid S \circ T \rangle$.

Case $\langle A \Vdash P \rangle \xrightarrow{\text{op}}_{\text{ci}} \langle A' \Vdash P' \rangle$.

Then $\xi_a = \emptyset$ and $\delta_p = \emptyset$ and there exists $N = \langle u \mid S' \circ T' \rangle$ and Σ'_a s.t. $(M \mid \Sigma_a) \mapsto_{\text{op}} (N \mid \Sigma'_a)$, $A' = \langle (N \mid \Sigma'_a); \gamma_p; \emptyset \rangle$ and $P' = P$.

Subcase $\Sigma'_a \cap \Sigma_p = \emptyset$.

It is immediate that $\text{supp}(T') \subseteq \text{supp}(M) \subseteq \text{dom}(\gamma_a) \cup \text{dom}(\delta_p)$. We have $\text{supp}(u), \text{supp}(S') \subseteq \text{supp}(t) \cup \text{supp}(S)$, then by hypothesis we get $\text{supp}(u), \text{supp}(S') \subseteq \text{supp}(\text{dom}(\gamma_a))$.

W.l.o.g. suppose $\exists T = \langle \Gamma_o; \emptyset \mid \Gamma_p; \emptyset \rangle$ and $\subseteq \text{dom}(\gamma_a)$. $P \circ \langle \Gamma_o; \emptyset \mid \Gamma_p \cdot [c \mapsto \neg\tau]; \emptyset \rangle$ and $A \circ T^\perp$, then we have $\Sigma_a; \Gamma_o \Vdash_c M$ and from lemma ??, we have $\Sigma'_a; \Gamma_o \Vdash_c N$, thus $A' \circ T^\perp$. The remaining complementarity conditions are trivially satisfied.

Case $\langle A \Vdash P \rangle \xrightarrow{\text{hdl}}_{\text{ci}} \langle A' \Vdash P' \rangle$.

Then $A' = \langle (M \mid \Sigma_a); \gamma_p; \emptyset \rangle$ and $P' = \langle \Sigma_p; \gamma_a \cdot \delta_a \mid \emptyset \rangle$ where $\xi_a = (\rightarrow, \rightarrow, \delta_a)$.

Since $\text{supp}(M) \subseteq \text{dom}(\gamma_a) \cup \text{dom}(\delta_a)$ then $\text{supp}(M) \subseteq \text{dom}(\gamma_a \cdot \delta_a)$.

W.l.o.g. let $T = \langle \Gamma_o; \Delta_o \mid \Gamma_p; \Delta_p \rangle$ and suppose $P \circ \langle \Gamma_o; \Delta_o \mid \Gamma_p \cdot [c \mapsto \neg\tau]; \Delta_p \rangle^\perp$ and $A \circ T$. Taking $S = \langle \Gamma_o \cdot \Delta_o; \emptyset \mid \Gamma_p; \emptyset \rangle$, since $\Sigma_a; \Gamma_o \cdot \Delta_o \Vdash_c M$, we can verify that $A' \circ S$ and $P \circ \langle \Gamma_o \cdot \Delta_o; \emptyset \mid \Gamma_p \cdot [c \mapsto \neg\tau]; \emptyset \rangle^\perp$.

Case $\langle A \Vdash P \rangle \xrightarrow{\text{dual}}_{\text{ci}} \langle A' \Vdash P' \rangle$.

By writing **abstract** $(M, \delta_p) \not\approx (\alpha.p, \gamma_{\text{abs}}, \xi_p)$ and **concretize** $(\alpha.p, \gamma_a; \xi_a) = (N, \delta_a)$ with $N = \langle u \mid S' \circ T' \rangle$, we have $A' = \langle (N \mid \Sigma_p); \gamma_a; \delta_a \rangle$ and $P' = \langle \Sigma_a; \gamma_p \cdot \gamma_{\text{abs}}; \delta_p \cdot \xi \rangle$.

We have $\text{supp}(\text{codom}(\delta_a)) = \text{supp}(\text{codom}(\xi_a))$ and $\text{supp}(\text{codom}(\xi)) \subseteq \text{supp}(t)$, thus by hypothesis, we have $\text{supp}(\text{codom}(\delta_a)) \subseteq \text{dom}(\gamma_p) \subseteq \text{dom}(\gamma_p \cdot \gamma_{\text{abs}})$, and we also have $\text{supp}(\text{codom}(\xi \cdot \delta_p)) \subseteq \text{supp}(\gamma_a)$.

W.l.o.g. let $T = \langle \Gamma_o; \Delta_o \mid \Gamma_p; \Delta_p \rangle$ and suppose $P \circ \langle \Gamma_o; \Delta_o \mid \Gamma_p \cdot [c \mapsto \neg\tau]; \Delta_p \rangle^\perp$ and $A \circ T$. Since $\Sigma_a; \Gamma_o \cdot \Delta_o \Vdash_c M$ and $\alpha \in \text{dom}(\Gamma_o)$ (because $\alpha \in \text{supp}(t) \cup \text{supp}(S)$), then there exists τ s.t. $\Gamma_o(\alpha) = \tau$.

By writing $\Gamma_o \cdot \Delta_o \Vdash p : \neg\tau \triangleright \Gamma_p; \Delta_p$ and taking $\mathbb{S} = \langle \Gamma_p \cdot \Gamma_p; \Delta_p \cdot \Delta_p \mid \Gamma_o; \Delta_o \rangle$, it is easy to verify that $A' \circ \mathbb{S}$ and $P' \circ \langle \Gamma_o; \Delta_o \mid \Gamma_p \cdot \Gamma_p \cdot [c \mapsto \neg\tau]; \Delta_p \cdot \Delta_p \rangle$.

□

B.2.2 Concrete observation

If an observation amounts to evaluating an observable value, an environment configuration can only do this after a switch of perspective, i.e through a $\xrightarrow{\text{dual}}_{\text{ci}}$ transition. For this reason we will define the observation transition $\Downarrow_{\text{ci}}$ a closure on $\xrightarrow{\text{dual}}_{\text{ci}}$ (up-to $\xrightarrow{\text{eval}}_{\text{ci}}$ and $\xrightarrow{\text{hdl}}_{\text{ci}}$) such that $\mathbb{C} \Downarrow_{\text{ci}} \mathbb{D}$ means $\mathbb{C} \mapsto_{\text{ci}}^* \mathbb{D}$ where $\mathbb{C}$ and $\mathbb{D}$ are of opposite perspective.

Definition B.3 Taking $\Rightarrow_{\text{dual}}$ to be $(\xrightarrow{\text{hdl}}_{\text{ci}} \cup \xrightarrow{\text{eval}}_{\text{ci}})^* \xrightarrow{\text{dual}}_{\text{ci}} (\xrightarrow{\text{hdl}}_{\text{ci}} \cup \xrightarrow{\text{eval}}_{\text{ci}})^*$ we define:

$$\mathbb{C} \Downarrow_{\text{ci}} \mathbb{D} :\iff \exists k \in \mathbb{N}. \mathbb{C} \Rightarrow_{\text{dual}}^{2*k+1} \mathbb{D} \wedge \mathbb{D} \not\Downarrow_{\text{ci}}$$

where, for $n \in \mathbb{N}$, $\Rightarrow_{\text{dual}}^n := \overbrace{\Rightarrow_{\text{dual}} \cdots \Rightarrow_{\text{dual}}}^{n \text{ times}}$.

B.2.3 Relating $\mathcal{L}_{\text{AI}}$ and $\mathcal{L}_{\text{ci}}$

We observe that, by definition, the *interaction* between a program and its environment given in the form of *concrete interaction* hides the *parallel abstract composition* of the underlying complementary passive and active $\mathcal{L}_{\text{AI}}$ configurations.

The following proposition captures how the $\mapsto_{\text{ci}}$ transitions, can be equivalently re-expressed in terms of *complementarity* and the transition $\rightarrow_{\text{I}}$.

Proposition B.4 (Parallel composition)

$$\begin{array}{ccc} \text{EVALUATING} & \text{HANDLING} & \text{DUALIZING} \\ \frac{\mathbb{J} \dashv \vdash \mathbb{I} \quad \mathbb{I} \xrightarrow{\text{eval}}_{\text{I}} \mathbb{J}}{\langle \mathbb{J} \dashv \vdash \mathbb{I} \rangle \xrightarrow{\text{eval}}_{\text{ci}} \langle \mathbb{J} \dashv \vdash \mathbb{J} \rangle} & \frac{\mathbb{J} \dashv \vdash \mathbb{I} \quad \mathbb{J} \xrightarrow{\text{hdl}^\ominus}_{\text{I}} \mathbb{K} \quad \mathbb{I} \xrightarrow{\text{hdl}^\oplus}_{\text{I}} \mathbb{J}}{\langle \mathbb{J} \dashv \vdash \mathbb{I} \rangle \xrightarrow{\text{hdl}}_{\text{ci}} \langle \mathbb{J} \dashv \vdash \mathbb{J} \rangle} & \frac{\mathbb{J} \dashv \vdash \mathbb{I} \quad \mathbb{J} \xrightarrow{\text{m}}_{\text{I}} \mathbb{J} \quad \mathbb{I} \xrightarrow{\text{m}^\perp}_{\text{I}} \mathbb{K}}{\langle \mathbb{J} \dashv \vdash \mathbb{I} \rangle \xrightarrow{\text{dual}}_{\text{ci}} \langle \mathbb{K} \dashv \vdash \mathbb{J} \rangle} \end{array}$$

Corollary B.5 Given $\mathbb{I} \in \text{Confs}_{\text{act}}$ and $\mathbb{J} \in \text{Confs}_{\text{pas}}$ such that $\mathbb{I} \dashv \vdash \mathbb{J}$, we have:

$$\langle \mathbb{J} \dashv \vdash \mathbb{I} \rangle \mapsto_{\text{ci}}^* \langle \mathbb{J}_0 \dashv \vdash \mathbb{J}_1 \rangle \iff \exists t \in \mathbf{Tr}_{\text{AI}}, i \in \{0, 1\}. \mathbb{J} \xRightarrow{\text{t}}_{\text{I}} \mathbb{J}_0 \wedge \mathbb{I} \xRightarrow{\text{t}^\perp}_{\text{I}} \mathbb{J}_1$$

where $\xRightarrow{\text{m}}_{\text{I}} := \xrightarrow{\text{hdl}}_{\text{I}} \xrightarrow{\text{eval}}_{\text{I}}^* \xrightarrow{\text{m}}_{\text{I}} \xrightarrow{\text{hdl}}_{\text{I}} \xrightarrow{\text{eval}}_{\text{I}}^*$

C Adequacy

Now that all the necessary definitions are available, we will state the main intermediate lemmas for proving adequacy: lemmas C.1 and C.2, and we will prove them in the following sections D and E, respectively.

Lemma C.1 (Bisimilarity) *Taking an interactive term $\Sigma_1; \Gamma \Vdash_c M$ and a compatible name assignment $\Sigma'_1; d : \neg \mathbb{1} \vdash \gamma : \Gamma$, then:*

$$\begin{aligned} \langle \Sigma'_1, \gamma; \perp \Vdash (M \mid \Sigma_1); \varepsilon, \perp \rangle &\Downarrow_{\text{ci}} \langle \Sigma_2; \gamma_2; \perp \Vdash (d[\mathbf{return} ()] \mid \Sigma'_2); \gamma_1; \perp \rangle \\ &\iff \\ (M\{\gamma\} \mid \Sigma'_1 \cdot \Sigma_1) &\Downarrow_{\text{op}} (d[\mathbf{return} ()] \mid \Sigma'_2 \cdot \Sigma_2) \end{aligned}$$

Lemma C.2 (Full observation) *Taking an interactive term $\Sigma; \Gamma \Vdash_c M$ and a compatible name assignment $\Sigma'; d : \neg \mathbb{1} \vdash \gamma : \Gamma$, then by writing $\mathbb{G} = \langle I \parallel T \parallel W \rangle$ and $\mathbb{H} = \langle J \parallel S \parallel U \rangle$ for their respective **ogs** embeddings:*

$$\begin{aligned} \exists \mathbb{K}, \gamma'. \langle J \Vdash I \rangle &\Downarrow_{\text{ci}} \langle \mathbb{K} \Vdash d[\mathbf{return} ()]; \gamma'; \perp \rangle \\ &\iff \\ \exists t \in \mathbf{CTr}_{\text{ogs}}(\mathbb{G}). t^\perp d.\langle \mathbf{ret} () \mid \square \rangle^\oplus &\in \mathbf{CTr}_{\text{ogs}}(\mathbb{H}) \end{aligned}$$

Theorem C.3 (Adequacy) *Given an interactive term $\Sigma; \Gamma \Vdash_c M$ and a compatible name assignment $\Sigma'; d : \neg \mathbb{1} \vdash \gamma : \Gamma$, then by writing $\mathbb{P}$ and $\mathbb{E}$ for their respective **ogs** embeddings, we have:*

$$\begin{aligned} (M\{\gamma\} \mid \Sigma' \cdot \Sigma) &\Downarrow_{\text{op}} (d[\mathbf{return} ()] \mid \Sigma'') \\ &\iff \\ \exists t \in \mathbf{CTr}_{\text{ogs}}(\mathbb{P}). t^\perp d.\langle \mathbf{ret} () \mid \square \rangle^\oplus &\in \mathbf{CTr}_{\text{ogs}}(\mathbb{E}) \end{aligned}$$

Proof. Follows from the two preceding lemmas. □

D Bisimilarity (proof of lemma C.1)

$(\mathcal{L}_{\text{ci}}, \mapsto_{\text{ci}})$ and $(\Lambda_{\text{eff}}, \mapsto_{\text{op}})$ are weakly bisimilar

We start by stating the main result of this section; that is modulo $\xrightarrow{\text{ci}}^{\text{dual}}$ and $\xrightarrow{\text{ci}}^{\text{hdl}}$ reductions, the transition systems $(\mathcal{L}_{\text{ci}}, \mapsto_{\text{ci}})$ and $(\Lambda_{\text{eff}}, \mapsto_{\text{op}})$ are related by a weak bisimulation. Afterwards we will introduce the necessary definitions and lemmas in order to prove it.

The proof relies on a notion of *telescoped substitution* to avoid cycles in the concatenation of mappings coming from the composition of two interactive configurations. This enforces the absence of livelocks in the interaction.

Definition D.1 A *telescoped substitution* δ is a substitution seen as a stack of mappings $[\alpha_0 \mapsto v_0, \dots, \alpha_k \mapsto v_k]$ such that for all $i \in \{1, \dots, k\}$, we have $\text{supp}(v_i) \cap \{\alpha_i, \dots, \alpha_k\} = \emptyset$.

Proposition D.2 Given a telescoped substitution $[\alpha_0 \mapsto v_0, \dots, \alpha_k \mapsto v_k]$, for all $i \in \{1, \dots, k\}$, we have $\text{supp}(v_i) \subseteq \{\alpha_0, \dots, \alpha_i\}$.

A telescoped substitution δ can be transformed into a substitution δ^* such that for any $\alpha \in \text{dom}(\delta)$ $\text{supp}(\delta(\alpha)) \subseteq \text{supp}(v_0)$.

Definition D.3 Taking a telescoped substitution δ and $k \in \mathbb{N}^*$, we define the iterated telescoped substitution δ^i as:

$$\delta^1 := \delta \quad \delta^{i+1} := \Pi_{n \in \text{dom}(\delta)} [n \mapsto \delta(n)\{\delta^i\}]$$

We then define δ^* as δ^k with k the size of the domain of δ . Then if $\Gamma \cdot \Delta \vdash \delta : \Gamma$ and $\text{dom}(\Delta) = \text{supp}(v_0)$, we have $\Delta \vdash \delta^* : \Gamma$.

Lemma D.4 Taking a telescoped substitution δ and $\alpha \in \text{dom}(\delta)$, then there exists a name $\alpha \in \text{dom}(\delta)$ and a natural number $k \in \mathbb{N}^*$ such that $\text{supp}(\delta^k(\alpha)) \cap \text{dom}(\delta) = \emptyset$.

We define a bisimulation between $\mapsto_{\text{ci}}$ and $\mapsto_{\text{op}}$ by collapsing $\mathcal{L}_{\text{ci}}$ configurations into operational ones.

Lemma D.5 Let $(\mathbb{E} \Vdash \mathbb{P}) \in \text{Confs}_{\text{ci}}$ such that $\mathbb{E} \Vdash_d \mathbb{P}$. If we write $\mathbb{E} = \langle \Sigma_p; \gamma_p; \xi_p \rangle$ and $\mathbb{P} = \langle (M \mid \Sigma_a); \gamma_a; \delta_a \rangle$ and $\mathbb{P} \circ (\Gamma_o; \Delta_o \mid \Gamma_p; \Delta_p)$, then $\delta = \gamma_a \cdot \gamma_p$ is a telescoped substitution, and there exists $\Sigma_a \cdot \Sigma_p; d : \neg \mathbb{1} \vdash \delta^* : \Gamma_p \cdot \Gamma_o$.

Definition D.6 We define the function $\Phi_{\text{op}}^{\text{ci}} : \text{Confs}_{\text{ci}} \rightarrow \Lambda_{\text{eff}}$ as the function:

$$\langle \Sigma_p; \gamma_p; \xi_p \Vdash (M \mid \Sigma_a); \gamma_a; \delta_a \rangle \mapsto (M\{(\gamma_p \cdot \gamma_a)^* \cdot \delta_e\} \mid \Sigma_p \cdot \Sigma_a)$$

with

$$\delta_e := \begin{cases} \varepsilon & \text{if } \xi_p = \perp \\ \delta_a \cdot \delta_p & \text{if for some } Y \in \{a, p\}, \xi_p = !(e, \neg, \delta_p) \end{cases}$$

We write $\mathcal{B}_{\text{op}}^{\text{ci}}$ for the functional relation corresponding to $\Phi_{\text{op}}^{\text{ci}}$.

Lemma D.7 If $\mathbb{C} \xrightarrow{\text{ci}}^{\text{dual}} \mathbb{D}$ or $\mathbb{C} \xrightarrow{\text{ci}}^{\text{hdl}} \mathbb{D}$, then $\Phi_{\text{op}}^{\text{ci}}(\mathbb{C}) = \Phi_{\text{op}}^{\text{ci}}(\mathbb{D})$.

Proof. We proceed by case analysis knowing that $\mathbb{C} = \langle \Sigma_p; \gamma_p; \xi_p \Vdash (P(\alpha) \mid \Sigma_a); \gamma_a; \delta_a \rangle$ and that $P(\alpha)$ is in normal form. $\square$

Lemma D.8 Taking a configuration $\mathbb{C} \in \text{Confs}_{\text{ci}}$, there exists a configuration $\mathbb{D}$ such that $\mathbb{C} \xrightarrow{\text{ci}}^{\text{dual}} \mathbb{D}$ and $\mathbb{D} \not\xrightarrow{\text{ci}}^{\text{dual}}$.

Proof. Let's write $\mathbb{C}$ as $\langle \Sigma_p; \gamma_p; \xi_p \Vdash (M \mid \Sigma_a); \gamma_a; \delta_a \rangle$ and $\gamma = [\alpha_0 \mapsto v_0, \dots, \alpha_n \mapsto v_n]$ for $\gamma_a \cdot \gamma_p$.

If M is not in normal form then $\mathbb{C} \not\xrightarrow{\text{ci}}^{\text{dual}}$, thus $\mathbb{D} = \mathbb{C}$. Otherwise, there exists an index $k \in \{0, \dots, n\}$ and a co-pattern, whose shape is either $\langle \square \mid K \rangle$ or $\langle _ \mid K[\square] \circ _ \rangle$, such that $M = P(\alpha_k)$.

We proceed by induction on k and on the length of K .

case $M = \langle f \ v \mid K \rangle$ with $f \in \text{dom}(\gamma)$.

Subcase $\gamma(f) \in \text{dom}(\gamma)$.

Since γ is a telescoped substitution, there exists $\ell \in \{k+1, \dots, n\}$ such that $\gamma(f) = \alpha_\ell$. Thus there exists a configuration $\mathbb{D}$ such that $\mathbb{C} \xrightarrow{\text{dual}}_{\text{ci}} \mathbb{C}'$ and whose active component can be written as $\langle d[\alpha_\ell A]; \gamma_P \cdot \gamma_A; \emptyset \rangle$. By induction hypothesis, there exists $\mathbb{D}$ such that $\mathbb{C}' \xrightarrow{\text{dual}}_{\text{ci}}^* \mathbb{D}$, and in particular $\mathbb{C}' \xrightarrow{\text{dual}}_{\text{ci}}^* \mathbb{D}$, and $\mathbb{D} \not\xrightarrow{\text{dual}}_{\text{ci}}$.

Subcase $\gamma(f) \notin \text{dom}(\gamma)$.

$\gamma(f)$ is a λ -abstraction. Then there exists a configuration $\mathbb{D}$ such that $\mathbb{C} \xrightarrow{\text{dual}}_{\text{ci}} \mathbb{D}$ and whose active component can be written as $\langle d[(\lambda x. t) A]; \gamma_P \cdot \gamma_A; \emptyset \rangle$. From which we can deduce that $\mathbb{D} \xrightarrow{\text{op}}_{\text{ci}}$ and in particular that $\mathbb{D} \not\xrightarrow{\text{dual}}_{\text{ci}}$.

case $M ::= \langle \text{return } v \mid K[\kappa] \rangle \mid \langle \underline{e} \mid K[\kappa] \circ S \rangle$ with $\kappa \in \text{dom}(\gamma)$.

Subcase $\gamma(\kappa) = T[E]$.

Then there exists a configuration $\mathbb{D}$ such that $\mathbb{C} \xrightarrow{\text{dual}}_{\text{ci}} \mathbb{D}$ and whose active term N can be written as $\langle \text{return } A \mid K[T[E]] \rangle$ in which case $N \mapsto_{\text{eval}}$, or as $\langle \underline{e}\{\delta_P\} \mid K[T[E]] \circ S \rangle$ in which case $N \mapsto_{\text{fwd}}$. In either case, we have $\mathbb{D} \xrightarrow{\text{op}}_{\text{ci}}$ and in particular that $\mathbb{D} \not\xrightarrow{\text{dual}}_{\text{ci}}$.

Subcase $\gamma(\kappa) = \square$.

Suppose $M = \langle \text{return } v \mid K[\kappa] \rangle$, we have

$$\begin{aligned} \mathbb{C} &\xrightarrow{\text{dual}}_{\text{ci}} \langle \Sigma_a; \gamma_a; \emptyset \vdash \langle \text{return } A \mid d[] \mid \Sigma_P \rangle; \gamma_P \cdot \gamma_A \cdot [d \mapsto K]; \emptyset \rangle \\ &\xrightarrow{\text{dual}}_{\text{ci}} \langle \Sigma_P; \gamma_P \cdot \gamma_A \cdot \gamma_d; \emptyset \vdash \langle \text{return } B \mid K \mid \Sigma_a \rangle; \gamma_a \cdot \gamma_B; \emptyset \rangle = \mathbb{C}' \end{aligned}$$

We conclude by applying the induction hypothesis on $\mathbb{C}'$.

The same reasoning applied to the other case.

case $M ::= \langle \text{return } v \mid c[] \rangle \mid \langle \underline{e} \mid c[] \circ S \rangle$.

Subcase $c \notin \text{dom}(\gamma)$. Then $\mathbb{D} = \mathbb{C}$ since $\mathbb{C} \not\xrightarrow{\text{dual}}_{\text{ci}}$.

Subcase $\gamma(c) ::= \gamma(c) = \alpha_\ell[] \mid K[\alpha_\ell[]]$ with $\ell \in \{k+1, \dots, n\}$.

There exists a configuration $\mathbb{C}'$ such that $\mathbb{C} \xrightarrow{\text{dual}}_{\text{ci}} \mathbb{C}'$ and whose active component can be written as $\langle (Q(\alpha_\ell) \mid \Sigma_P); \gamma_P \cdot \gamma_Q; \emptyset \rangle$ where Q is a co-pattern. We conclude using the induction hypothesis on $\mathbb{C}'$.

□

Lemma D.9 Let $\Sigma; \Gamma \Vdash_{\text{c}} M$ be an interactive computation and let $\Sigma; \Delta \vdash \gamma : \Gamma$ be a name assignment. If $(M \mid \Sigma) \mapsto_{\text{op}} (N \mid \Sigma')$ then $(M\{\gamma\} \mid \Sigma) \mapsto_{\text{op}} (N\{\gamma\} \mid \Sigma')$.

Lemma D.10 Let $\Sigma; \Gamma \Vdash_{\text{c}} M$ be a computation s.t. $(M \mid \Sigma) \mapsto_{\text{op}}$ and let $\Sigma; \Delta \vdash \gamma : \Gamma$ be a name assignment. If $(M\{\gamma\} \mid \Sigma) \mapsto_{\text{op}} (N\{\gamma\} \mid \Sigma')$ and $\text{supp}(M) = \text{supp}(N)$, then $(M \mid \Sigma) \mapsto_{\text{op}} (N \mid \Sigma')$.

Proof. By case analysis knowing that any such term M is given by the syntax:

$$\begin{aligned} \text{Nf}_{\text{fwd}} ::= & \langle \text{return } v \mid K[E] \rangle \mid \langle (\lambda x. t) v \mid K \rangle \\ & \mid \langle \iota \# \text{op } v \mid K[\{\}]\text{ with } H \rangle \circ S \quad (\iota \in \text{hdl}(H)) \\ & \mid \langle \underline{e} \mid K[E] \circ S \rangle \end{aligned}$$

□

Lemma D.11 Taking a configuration $\mathbb{C} \in \text{Confs}_{\text{ci}}$ s.t. $\Phi_{\text{op}}^{\text{ci}}(\mathbb{C}) \mapsto_{\text{op}} (N \mid \Sigma)$, then there exists a configuration $\mathbb{D} \in \text{Confs}_{\text{ci}}$ s.t. $\mathbb{C} \xrightarrow{\text{ci}} \mathbb{D}$ and $\Phi_{\text{op}}^{\text{ci}}(\mathbb{D}) = (N \mid \Sigma)$.

Proof. Let's write $\mathbb{C} = \langle \Sigma_P; \gamma_P; \xi_P \vdash (M \mid \Sigma_a); \gamma_a; \delta_a \rangle$.

Case $\mathbb{C} \not\mapsto_{\text{ci}}^{\text{dual}}$; that is $\mathbb{M} \notin \text{Nf}(\Lambda_{\text{eff}})$.

By hypothesis, we have $\Phi_{\text{op}}^{\text{ci}}(\mathbb{C}) = (\mathbb{M}\{\gamma\} \mid \Sigma_p \cdot \Sigma_a) \mapsto_{\text{op}} (\mathbb{N}'\{\gamma\} \mid \Sigma)$ where $\gamma = (\gamma_p \cdot \gamma_a)^*$ and $\mathbb{N}'$ is such that $\text{supp}(\mathbb{M}) = \text{supp}(\mathbb{N}')$ and $\mathbb{M} = \mathbb{N}'\{\gamma\}$. Then from lemma D.10, we can deduce that $\mathbb{C} \mapsto_{\text{ci}}^{\text{op}} \langle \Sigma_p; \gamma_p; \emptyset \vdash (\mathbb{N}' \mid \Sigma'_a); \gamma_a; \emptyset \rangle = \mathbb{D}$ where $\Sigma = \Sigma_p \cdot \Sigma'_a$, which entails that $\Phi_{\text{op}}^{\text{ci}}(\mathbb{D}) = (\mathbb{N}'\{\gamma\} \mid \Sigma_p \cdot \Sigma'_a) = (\mathbb{N} \mid \Sigma)$.

Case $\mathbb{C} \mapsto_{\text{ci}}^{\text{dual}}$.

By lemma D.8, there exists $\mathbb{C}'$ s.t. $\mathbb{C} \mapsto_{\text{ci}}^{\text{dual}} \mathbb{C}'$ and $\mathbb{C}' \not\mapsto_{\text{ci}}^{\text{dual}}$. Since $\Phi_{\text{op}}^{\text{ci}}(\mathbb{C}) = \Phi_{\text{op}}^{\text{ci}}(\mathbb{C}')$ (by lemma D.7), then by applying the above argument to $\mathbb{C}'$ we can prove what we want. $\square$

Lemma D.12 (bisimulation) Taking $\Rightarrow_{\text{ci}}^{\text{op}} := (\mapsto_{\text{ci}}^{\text{hdl}})^* \mapsto_{\text{ci}}^{\text{op}}$ and $\Rightarrow_{\text{ci}} := (\mapsto_{\text{ci}}^{\text{dual}})^* \mapsto_{\text{ci}}^{\text{op}}$, there exists a bisimulation relating $(\text{Confs}_{\text{ci}}, \Rightarrow_{\text{ci}})$ and $(\Lambda_{\text{eff}}, \mapsto_{\text{op}})$, i.e there exists a relation $\mathcal{B}_{\text{op}}^{\text{ci}} \in \wp(\text{Confs}_{\text{ci}} \times \Lambda_{\text{eff}})$ s.t. the following diagram holds.

$$\begin{array}{ccc} \mathbb{C} & \mathcal{B}_{\text{op}}^{\text{ci}} & \mathbb{M} \\ \text{ci} \downarrow & & \downarrow \text{op} \\ \mathbb{D} & \mathcal{B}_{\text{op}}^{\text{ci}} & \mathbb{N} \end{array}$$

Proof. We show that $\mathcal{B}_{\text{op}}^{\text{ci}}$ and its converse are simulations.

$$\begin{array}{ccccc} \mathbb{C} & \mathcal{B}_{\text{op}}^{\text{ci}} & \mathbb{M} & \mathbb{C} & \mathcal{B}_{\text{op}}^{\text{ci}} & \mathbb{M} \\ \downarrow & & & \downarrow & & \\ \text{ci} \downarrow & \text{lemmas (D.8)/(D.7)} & = & \text{ci} \downarrow & \text{lemmas (D.8)/(D.7)} & = \\ \text{dual} & & & \text{dual} & & \\ * & & & * & & \\ \mathbb{C}' & \mathcal{B}_{\text{op}}^{\text{ci}} & \mathbb{M} & \exists \mathbb{C}' & \mathcal{B}_{\text{op}}^{\text{ci}} & \mathbb{M} \\ \downarrow & & \downarrow \text{op} & \downarrow & & \downarrow \text{op} \\ \text{ci} \downarrow & \text{lemmas (D.7)/(D.9)} & & \text{ci} \downarrow & \text{lemmas (D.7)/(D.10)} & \\ \text{op} & & \exists \mathbb{N} & \text{op} & & \mathbb{N} \\ \mathbb{D} & \mathcal{B}_{\text{op}}^{\text{ci}} & & \exists \mathbb{D} & \mathcal{B}_{\text{op}}^{\text{ci}} & \end{array}$$

$\square$

Corollary D.13 Taking a nominal term $\Sigma; \Gamma; \emptyset \vdash_c \mathbf{t} : \tau$, a compatible evaluation context $\Sigma'; \Delta \vdash \mathbf{E} : \tau \rightsquigarrow \mathbb{1}$ and a assignment $\Sigma'; \Delta \vdash \gamma : \Gamma$, then:

$$\begin{aligned} \langle \Sigma'_1, \gamma \cdot [c \mapsto d[\mathbf{E}]]; \perp \vdash (c[\mathbf{t}] \mid \Sigma_1); \varepsilon, \perp \rangle \Downarrow_{\text{ci}} \langle \Sigma_2; \gamma_2; \perp \vdash (d[\mathbf{return}()] \mid \Sigma'_2); \gamma_1; \perp \rangle \\ \iff \\ \langle \mathbf{E}[\mathbf{t}\{\gamma\}] \mid \Sigma'_1 \cdot \Sigma_1 \rangle \Downarrow_{\text{op}} \langle d[\mathbf{return}()] \mid \Sigma'_2 \cdot \Sigma_2 \rangle \end{aligned}$$

E Semantic observation (proof of lemma C.2)

E.1 Relating $(\mathcal{L}_T, \rightarrow_T)$ to $(\mathcal{L}_{AI}, \rightarrow_I)$

Lemma E.1 *Taking a well-typed interactive term in normal form $\Sigma; \Gamma \Vdash_c \text{Nf}$ such that $\text{abstract}(\text{Nf}, \delta) \not\approx (\alpha.q, \gamma, \delta.\xi)$ and $\Gamma(\alpha) = \tau$ and $\Gamma \Vdash q : \neg\tau \triangleright \Gamma_q; \Delta_q$, we have: $\Gamma \vdash \gamma : \Gamma_q$ and $\Gamma \vdash \xi : \Delta_q$.*

Lemma E.2 *Given a well-typed interactive term $\Sigma; \Delta \cdot \Gamma \Vdash_c M$ and a substitution $\Gamma' \vdash \delta : \Delta$ such that $\Gamma' \subseteq \Gamma$ and $\text{dom}(\Delta) \subseteq \text{dom}(\delta)$, then: $\Sigma; \Gamma \Vdash_c M\{\delta\}$*

Lemma E.3 (Subject reduction) *Taking $\mathbb{I}, \mathbb{J} \in \mathcal{A}$ and $\mathbb{T} \in \mathcal{T}$ such that $\mathbb{I} \Vdash \mathbb{T}$ and $\mathbb{I} \xrightarrow{1} \mathbb{J}$, then there exists $\mathbb{T} \in \mathcal{T}$ such that $\mathbb{T} \xrightarrow{\alpha}_T \mathbb{S}$ and $\mathbb{J} \Vdash \mathbb{S}$.*

Proof. We write $\mathbb{T} = \langle \Gamma_o; \Delta_o \mid \Gamma_p; \Delta_p \rangle$ and proceed by case analysis on α .

Case $\alpha = hdl^\oplus$.

Then if we take $\mathbb{I} = \langle (M \mid \Sigma); \gamma; !\delta \rangle$, we have $\mathbb{I} \xrightarrow{hdl^\oplus}_1 \langle (M \mid \Sigma); \gamma; \emptyset \rangle = \mathbb{J}$. It is immediate to verify that $\mathbb{S} = \langle \Gamma_o \cdot \Delta_o; \emptyset \mid \Gamma_p; \emptyset \rangle$ is s.t. $\mathbb{T} \xrightarrow{hdl^\oplus}_T \mathbb{S}$ and $\mathbb{J} \Vdash \mathbb{S}$.

Case $\alpha = hdl^\ominus$.

Then if we take $\mathbb{I} = \langle \Sigma; \gamma; ?\xi \rangle$ where $\xi = (-, -, \delta)$, we have $\mathbb{I} \xrightarrow{hdl^\ominus}_1 \langle \Sigma; \gamma \cdot \delta; \emptyset \rangle = \mathbb{J}$. Similarly, it is immediate to verify that $\mathbb{S} = \langle \Gamma_o; \emptyset \mid \Gamma_p \cdot \Delta_p; \emptyset \rangle$ is s.t. $\mathbb{T} \xrightarrow{hdl^\ominus}_T \mathbb{S}$ and $\mathbb{J} \Vdash \mathbb{S}$.

Case $\alpha = \alpha.p^\oplus$.

Then if we write $\mathbb{I} = \langle (\text{Nf} \mid \Sigma); \gamma; \delta \rangle$, we have $\mathbb{I} \xrightarrow{\alpha.p^\oplus}_1 \langle \Sigma; \gamma \cdot \gamma_{\text{abs}}; \xi \rangle = \mathbb{J}$ with $\text{abstract}(\text{Nf}, \delta) \not\approx (\gamma_{\text{abs}}, \delta \cdot \xi)$.

Since $\mathbb{I} \Vdash \mathbb{T}$, we have $\Sigma; \Gamma_o \cdot \Delta_o \Vdash_c \text{Nf}$, and since $\alpha \in \text{supp}(\text{Nf})$ then there exists τ, Γ_p, Δ_p such that $\Gamma_o(\alpha) = \tau$ and $\Gamma_o \cdot \Delta_p \cdot \Delta_o \Vdash p : \neg\tau \triangleright \Gamma_p; \Delta_p$, so by lemma E.1, we have $\Gamma_o \vdash \gamma_{\text{abs}} : \Gamma_p$ and $\Gamma_o \cdot \Delta_o \vdash \xi : \Delta_p$.

Thus by taking $\mathbb{S} = \langle \Gamma_o; \Delta_o \mid \Gamma_p \cdot \Gamma_p; \Delta_p \cdot \Delta_p \rangle$, it is immediate that $\mathbb{T} \xrightarrow{\alpha.p^\oplus}_T \mathbb{S}$ and $\mathbb{J} \Vdash \mathbb{S}$, knowing that $\Sigma; \Gamma_o \vdash \gamma : \Gamma_p$, and $\Sigma; \Gamma_o \vdash \delta : \Delta_p$.

Case $\alpha = \alpha.p^\ominus$.

Then if we take $\mathbb{I} = \langle \Sigma; \gamma; \xi \rangle$ where $\xi = (-, -, \delta)$, we have $\mathbb{I} \xrightarrow{\alpha.p^\ominus}_1 \langle (M \mid \Sigma); \gamma; \delta \rangle = \mathbb{J}$ where $M = p\{\delta\}(\gamma(\alpha))$.

Let τ, Γ_p, Δ_p such that $\Gamma_p(\alpha) = \tau$ and $\Delta_p \cdot \Delta_o \Vdash p : \neg\tau \triangleright \Gamma_p; \Delta_p$, then we have $\mathbb{T} \xrightarrow{\alpha.p^\ominus}_T \langle \Gamma_o \cdot \Gamma_p; \Delta_o \cdot \Delta_p \mid \Gamma_p; \Delta_p \rangle = \mathbb{S}$.

Since $\mathbb{I} \Vdash \mathbb{T}$, we have $\Sigma; \Gamma_o \vdash \gamma : \Gamma_p$ and $\Sigma; \Gamma_o \vdash \delta : \Delta_p$ and in particular $\Sigma; \Gamma_o \cdot \Gamma_p \vdash \gamma : \Gamma_p$ and $\Sigma; \Gamma_o \cdot \Gamma_p \vdash \delta : \Delta_p$.

Moreover, since $\Sigma; [\alpha \mapsto \tau] \cdot \Gamma_p \cdot \Delta_p \cdot \Delta_o \cdot \Delta_p \Vdash_c p(\alpha)$ and $\text{dom}(\Delta_o) = \text{dom}(\delta)$, then by lemma E.2 we have $\Sigma; [\alpha \mapsto \tau] \cdot \Gamma_p \cdot \Delta_o \cdot \Delta_p \Vdash_c p\{\delta\}(\alpha)$. Finally, given that $M = p\{\delta\}(\alpha)$ and $\Gamma_o \vdash \gamma(\alpha) : \tau$ then we can deduce that $\Sigma; \Gamma_o \cdot \Gamma_p \cdot \Delta_o \cdot \Delta_p \Vdash_c M$, and by extent $\mathbb{J} \Vdash \mathbb{S}$.

□

E.2 Compatibility of $(\mathcal{L}_{AI}, \rightarrow_I)$ and $(\mathcal{L}_{wb}, \rightarrow_{wb})$

Definition E.4 We define the predicate $\mathbb{M}_\square$ on $\{?, !\} \times \mathcal{N} \times F$, relating an *abstract forward stack* to a *call frame*, depending on which *player* has performed the effect.

$$\frac{\cdot \mathbb{M}_? []}{r \mathbb{M}_? f} \quad \frac{\cdot \mathbb{M}_! []}{r \mathbb{M}_! f}$$

$$\frac{}{r :: \kappa' :: \kappa \mathbb{M}_? \kappa :: f} \quad \frac{}{r :: \kappa' :: \kappa \mathbb{M}_! \kappa' :: f}$$

Definition E.5 [$(\mathcal{L}_{AI}, \mathcal{L}_{wb})$ -compatibility] Let $\mathbb{I}, \mathbb{J} \in \mathcal{A}$ and $\mathbb{W} \in \mathcal{W}$ s.t. $\mathbb{I} = \langle \gamma; \xi \rangle, \mathbb{J} = \langle \langle \mathbf{t} \mid \mathbf{K} \circ \mathbf{S} \mid \Sigma \rangle; \gamma'; \delta \rangle$ and $\mathbb{W} = \langle \pi \mid \eta \mid \phi \rangle$.

We say that $\mathbb{I}$ and $\mathbb{W}$ are compatible when $\mathbf{Compat}(\mathbb{I}, \mathbb{W})$ holds, where

$$\mathbf{Compat}(\mathbb{I}, \mathbb{W}) :\iff \forall \mathbf{f} \in \phi. \text{set}(\mathbf{f}) \subseteq \text{dom}(\gamma) \wedge \xi \sim_{\text{fwd}} \eta$$

Similarly, we say that $\mathbb{J}$ and $\mathbb{W}$ are compatible when $\mathbf{Compat}(\mathbb{J}, \mathbb{W})$ holds, where

$$\mathbf{Compat}(\mathbb{J}, \mathbb{W}) :\iff \forall \mathbf{f} \in \phi. \text{set}(\mathbf{f}) \subseteq \text{dom}(\gamma') \wedge \delta \sim_{\text{fwd}} \eta$$

with

$$\emptyset \sim_{\text{fwd}} \emptyset \quad !(\mathbf{e}, \rightarrow, -) \sim_{\text{fwd}} ([], \mathbf{e}) \quad ?(\mathbf{e}, r, -) \sim_{\text{fwd}} (f, \mathbf{e}) \text{ when } r \mathbb{A}_? f$$

$$!\delta^{\mathbf{e}} \sim_{\text{fwd}} ([], \mathbf{e}) \quad ?\delta \sim_{\text{fwd}} (f, \mathbf{e}) \text{ when } r \mathbb{A}_? f \text{ and } S\{\delta^{-1}\} = T \oplus r$$

Lemma E.6 (preservation of compatibility) *Let $\mathbb{I} \in \mathcal{A}$ and $\mathbb{W} \in \mathcal{W}$ s.t. $\mathbf{Compat}(\mathbb{I}, \mathbb{W})$.*

If $\mathbb{I} \xrightarrow{\alpha}_1 \mathbb{J}$ and $\mathbb{W} \xrightarrow{\alpha}_{wb} \mathbb{U}$, then $\mathbf{Compat}(\mathbb{J}, \mathbb{U})$.

Proof. We write $\mathbb{W} = \langle \pi \mid \eta \mid \phi \rangle$ with $\eta = (\mathbf{e}, f)$ then proceed by case analysis on $\mathbb{I}$ and α .

Case $\mathbb{I} = \langle \Sigma; \gamma; \blacksquare \xi \rangle$ with $\blacksquare \in \{?, !\}$ and $\xi = (\mathbf{e}, r, \delta)$.

Subcase $\alpha = hdl^{\ominus}$.

By hypothesis, we have $r \mathbb{A}_? f$ and $\forall f' \in \phi. \text{set}(f') \subseteq \text{dom}(\gamma)$. Moreover, we have $\mathbb{J} = \langle \Sigma; \gamma\delta; \emptyset \rangle$ and $\mathbb{U} = \langle \pi \mid \emptyset \mid \phi \cup \{f\} \rangle$, thus, since $\text{set}(f) \subseteq \text{dom}(\delta)$ we can deduce that $\forall f' \in \phi \cup \{f\}. \text{set}(f') \subseteq \text{dom}(\gamma \cdot \delta)$ and $\emptyset \sim_{\text{fwd}} \emptyset$, from which we can conclude that $\mathbf{Compat}(\mathbb{J}, \mathbb{U})$.

Subcase $\alpha ::= c.\langle \text{ret } A \mid \square \rangle^{\ominus} \mid \kappa.\langle \square[\text{ret } A] \mid d \rangle^{\ominus} \mid f.\langle \square A \mid d \rangle^{\ominus}$.

In either subcase, there exist M and π' s.t. $\mathbb{J} = \langle (M \mid \Sigma); \gamma; \emptyset \rangle$ and $\mathbb{U} = \langle \pi' \mid \emptyset \mid \phi \rangle$. Moreover, by definition $\emptyset \sim_{\text{fwd}} \emptyset$, and by hypothesis we have $\forall f \in \phi. \text{set}(f) \subseteq \text{dom}(\gamma)$, thus $\mathbf{Compat}(\mathbb{J}, \mathbb{U})$.

Subcase $\alpha ::= c.\langle \kappa[e] \mid \square \rangle^{\ominus} \mid \kappa'.\langle \square[\kappa[e]] \mid d \rangle^{\ominus}$ (performing an effect).

In either subcase, there exist K, π' s.t. $\mathbb{J} = \langle \langle \mathbf{e} \mid K \circ \kappa \mid \Sigma \rangle; \gamma; \varepsilon \rangle$ and $\mathbb{U} = \langle \pi' \mid (\mathbf{e}, []) \mid \phi \rangle$.

We have then, by definition, $\kappa \mathbb{A}_? []$ and $? \varepsilon \sim_{\text{fwd}} (\mathbf{e}, [])$, and by hypothesis, we have that $\forall f \in \phi. \text{set}(f) \subseteq \text{dom}(\gamma)$. Thus, we can conclude that $\mathbf{Compat}(\mathbb{J}, \mathbb{U})$.

Subcase $\alpha ::= c.\langle \kappa::r[e] \mid \square \rangle^{\ominus} \mid \kappa'.\langle \square[\kappa::r[e]] \mid d \rangle^{\ominus}$ (forwarding an effect).

In either subcase, there exist K, π' such that $\mathbb{J} = \langle \langle \mathbf{e}\{\delta\} \mid K \circ \kappa[r\{\delta\}] \mid \Sigma \rangle; \gamma; \blacksquare \delta \rangle$ and $\mathbb{U} = \langle \pi' \mid \eta \mid \phi \rangle$.

By hypothesis, we have that $\forall f \in \phi. \text{set}(f) \subseteq \text{dom}(\gamma)$ and $r \mathbb{A}_{\blacksquare} f$.

If $\blacksquare = ?$, then r is even-length, and by taking $S = \kappa[r\{\delta\}]$, we have $S\{\delta^{-1}\} = \kappa::r$ and $\kappa::r \mathbb{A}_? f$. If otherwise $\blacksquare = !$, then $\eta = \emptyset$.

Thus in both cases, we have $\blacksquare \delta \sim_{\text{fwd}} \eta$, and we can conclude that $\mathbf{Compat}(\mathbb{J}, \mathbb{U})$.

Case $\mathbb{I} = \langle \langle t \mid K \circ S \mid \Sigma \rangle; \gamma; \delta \rangle$.

Subcase $\alpha ::= d.\langle \text{ret } A \mid \square \rangle^{\oplus} \mid \kappa.\langle \square[\text{ret } A] \mid c \rangle^{\oplus} \mid f.\langle \square A \mid c \rangle^{\oplus}$.

In either subcase, there exist γ' and π' s.t. $\mathbb{J} = \langle \Sigma; \gamma \cdot \gamma'; \emptyset \rangle$ and $\mathbb{U} = \langle \pi' \mid \emptyset \mid \phi \rangle$. By definition, we have $\emptyset \sim_{\text{fwd}} \emptyset$, and by hypothesis we have $\forall f \in \phi. \text{set}(f) \subseteq \text{dom}(\gamma)$, and in particular $\forall f \in \phi. \text{set}(f) \subseteq \text{dom}(\gamma \cdot \gamma')$, thus $\mathbf{Compat}(\mathbb{J}, \mathbb{U})$.

Subcase $\alpha ::= d.\langle \kappa::r[e] \mid \square \rangle^{\oplus} \mid \kappa'.\langle \square[\kappa::r[e]] \mid c \rangle^{\oplus}$.

In either subcase, there exist γ', π', η', S' and T s.t. $\mathbb{J} = \langle \Sigma; \gamma \cdot \gamma'; \blacksquare \xi \rangle$ and $\mathbb{U} = \langle \pi' \mid \eta' \mid \phi \rangle$, where $S = T \oplus S'$, $S\{\delta^{-1}\} = r$ and $\xi = (\mathbf{e}, r, \delta \cdot [\kappa \mapsto T])$.

By hypothesis, we have that $\forall f \in \phi. \text{set}(f) \subseteq \text{dom}(\gamma)$ and $r \mathbb{A}_{\blacksquare} f$.

If $\blacksquare = ?$, then r is odd-length and $\eta' = (f \# [\kappa], \mathbf{e})$, thus $\kappa::r \mathbb{A}_? f \# [\kappa]$. If otherwise $\blacksquare = !$, then $\eta = \eta' = \emptyset$.

Thus in both cases, we have $\blacksquare \xi \sim_{\text{fwd}} \eta$, and we can conclude that $\mathbf{Compat}(\mathbb{J}, \mathbb{U})$.

□

E.3 Semantic ciu-observation

Given a term $\Sigma; \Gamma; \emptyset \vdash_{\mathcal{C}} \mathbf{t} : \tau$, an *observer* of $\mathbf{t}$ is an environment given by an evaluation context and a name valuation $(\mathbf{E}, \nu)$ such that $\Sigma'; \emptyset \vdash \mathbf{E} : \tau \rightsquigarrow \mathbb{1}$ and $\Sigma'; \emptyset \vdash \nu : \Gamma$.

We want to give a semantic characterization of "observing ()" but first we need to introduce the notion of *closed composability* of OGS configurations, which is the semantic counterpart of $\mathbf{t}$ being composable with $(\mathbf{E}, \nu)$ and $\mathbf{E}[\mathbf{t}\{v\}]$ being a closed term of type $\mathbb{1}$.

E.3.1 Closed composability of OGS configurations

If *complementarity* guarantees the "closedness" part of *closed composability*, we still require non-intial *program* and *environment* configurations to agree on their history of interaction. We will capture this *coherence of trajectory* by the following *synchronicity relation*.

First, we will the notion of *return frame* and auxiliary definitions.

Definition E.7 (return frames) We will call a *return frame* any control-flow stack of the shape $f \# [c]; i.e.$ that it is made up of a call frame ($f \in \text{List}(\mathcal{K})$) to which is appended a single abstract undelimited continuation ($c \in \mathcal{C}$).

We will also define the function F that, given an evaluation stack, computes the associated *return frame*.

$$F(c[S]) := F(S) \# [c] \quad F(\mathbf{T}[\kappa[S]]) := F(S) \# [\kappa] \quad F(\mathbf{T}) := []$$

Definition E.8 (return frames decomposition) For any a non-empty control-flow stack π generated by $\mathcal{L}_{wb}$, there corresponds a natural number $k \in \mathbb{N}^*$ and k *return frames* $\pi_1, \dots, \pi_k$ such that $\pi = \pi_1 \# \dots \# \pi_k$.

We will write $\pi_1 \oplus \dots \oplus \pi_k$ for this unique decomposition of π .

Definition E.9 (Synchronicity) We define synchronicity as a quaternary relation $\mathcal{S} \subseteq \mathcal{A} \times \mathcal{W} \times \mathcal{A} \times \mathcal{W}$.

Let $\mathbb{I} = \langle \mathbf{M} \mid \Sigma \rangle; \gamma_{\mathbb{I}}; \blacksquare \delta_{\mathbb{I}} \rangle$ in $\text{Confs}_{\text{act}}$, $\mathbb{J} = \langle \gamma_{\mathbb{J}}, \square(_, r, _) \rangle$ in $\text{Confs}_{\text{pas}}$ where $\mathbf{M} = \langle \mathbf{t} \mid c_0[S_0] \circ \mathbf{S} \rangle$ and $\blacksquare, \square \in \{?, !\}$, Let $\mathbb{W} = \langle \pi_{\mathbb{W}} \mid (f_{\mathbb{W}}, e_{\mathbb{W}}) \mid \phi_{\mathbb{W}} \rangle$, $\mathbb{U} = \langle \pi_{\mathbb{U}} \mid (f_{\mathbb{U}}, e_{\mathbb{U}}) \mid \phi_{\mathbb{U}} \rangle$ in $\mathcal{W}$ where $\pi_0 \oplus \dots \oplus \pi_k$ and $\pi'_0 \oplus \dots \oplus \pi'_p$ (for some $k, p \in \mathbb{N}^*$) are the respective decompositions of $\pi_{\mathbb{W}}$ and $\pi_{\mathbb{U}}$ into *return frames*.

We say that $\langle \mathbb{I}, \mathbb{W} \rangle$ and $\langle \mathbb{J}, \mathbb{U} \rangle$ are *in sync*, and write $\langle \mathbb{I}, \mathbb{W} \rangle \mathcal{S} \langle \mathbb{U}, \mathbb{J} \rangle$ when the following holds:

- $\mathbb{I} \# \mathbb{J}$ and for all $i \geq 0$, we can write $\gamma_{\mathbb{J}}(c_i) = d_{i+1}[S'_i]$ and $\gamma_{\mathbb{I}}(d_{i+1}) = c_{i+1}[S_{i+1}]$.
- $r \# \blacksquare f_{\mathbb{U}}$ and $r \# \square f_{\mathbb{J}}$
- $\forall 0 \leq i. F(c_i[S_i]) \simeq_{\phi_{\mathbb{U}}} \pi'_i$ and $F(\gamma_{\mathbb{J}}(c_i)) \simeq_{\phi_{\mathbb{W}}} \pi_i$

where $\simeq_{\phi}$ is given by

$$\text{REFL} \frac{\pi = \pi' \quad \phi \in K}{\pi \simeq_{\phi} \pi'} \quad \text{UP-TO} \frac{\pi = \pi' \quad \phi \in K \quad f \in \phi}{\pi \simeq_{\phi} f \# \pi'}$$

Next we show how the *synchronicity* relation indeed captures the *coherence* of both the *program's* and the *environment's* trajectories, in the sense that whatever interaction is triggered by the *active* player is in fact compatible with what the *passive* player is expecting, and that this property is preserved by interaction.

This is formalised in the next lemma, which proves that $\mathcal{S}$ is a simulation.

Lemma E.10 (O-simulation) Given $\mathbb{K} \in \text{Confs}_{\text{pas}}$, $\mathbb{I} \in \text{Confs}_{\text{act}}$, $\mathbb{W}, \mathbb{U} \in \mathcal{W}$ such that $\langle \mathbb{I}, \mathbb{W} \rangle \mathcal{S} \langle \mathbb{K}, \mathbb{U} \rangle$, we have:

$$\frac{\mathbb{I} \xrightarrow{\mathbf{m}}_1 \mathbb{J} \quad \mathbb{W} \xrightarrow{\mathbf{m}}_{wb} \mathbb{W}'}{\exists \mathbb{J}, \mathbb{U}'. \mathbb{K} \xrightarrow{\mathbf{m}^\perp}_1 \mathbb{J} \quad \mathbb{U} \xrightarrow{\mathbf{m}^\perp}_{wb} \mathbb{U}' \quad \langle \mathbb{J}, \mathbb{W}' \rangle \mathcal{S} \langle \mathbb{J}, \mathbb{U}' \rangle}$$

Proof. We write $\mathbb{I} = \langle \langle \mathbf{t} \mid c_0[S_0] \circ \mathbf{S} \mid \Sigma_{\mathbb{I}} \rangle; \gamma_{\mathbb{I}}, \delta_{\mathbb{I}} \rangle$, $\mathbb{W} = \langle \pi_{\mathbb{W}} \mid \eta_{\mathbb{W}} \mid \phi_{\mathbb{W}} \rangle$, $\mathbb{W}' = \langle \pi_{\mathbb{W}'} \mid (_, r_{\mathbb{W}'}, _) \mid \phi_{\mathbb{W}'} \rangle$, $\mathbb{U} = \langle \pi_{\mathbb{U}} \mid \eta_{\mathbb{U}} \mid \phi_{\mathbb{U}} \rangle$ and $\mathbb{K} = \langle \Sigma_{\mathbb{K}}; \gamma_{\mathbb{K}}; \xi_{\mathbb{K}} \rangle$. We will also write $\pi_0 \oplus \dots \oplus \pi_k$ and $\pi'_0 \oplus \dots \oplus \pi'_p$ (for some $k, p \in \mathbb{N}^*$)

for the respective decompositions of $\pi_{\mathbb{W}}$ and $\pi_{\mathbb{U}}$, as well as $\gamma_{\mathbb{K}}(c_i) = d_{i+1}[S'_i]$ and $\gamma_{\mathbb{I}}(d_{i+1}) = c_{i+1}[S_{i+1}]$ for $i \geq 0$.

We proceed by case analysis on $\mathbf{m}$.

Case $\mathbf{m} = \mathbf{c}_0.\langle \text{ret } A \mid \square \rangle^\oplus$.

We have $S_0 = c_0[\square]$, $\mathbf{S} = \square$, $\mathbf{t} = \text{return } V$, $\delta_{\mathbb{I}} = \emptyset$ and $\eta_{\mathbb{W}} = \emptyset$, $r_{\mathbb{W}'} = \cdot$.

$\mathbb{I}, \mathbb{W} \xrightarrow{\mathbf{m}} \langle \Sigma_{\mathbb{I}}; \gamma_{\mathbb{I}}; \emptyset \rangle, \langle \pi_{\mathbb{W}'} \mid \emptyset \mid \phi_{\mathbb{W}'} \rangle$

Since $\langle \mathbb{I}, \mathbb{W} \rangle \mathcal{S} \langle \mathbb{K}, \mathbb{U} \rangle$, then $\pi'_0 = F(c_0[\square]) = [c_0]$ and $\xi_{\mathbb{K}} = \emptyset$. Thus $\mathbb{U} \xrightarrow{\mathbf{m}^\perp}_{wb} \langle \pi_{\mathbb{U}'} \mid \emptyset \mid \phi_{\mathbb{U}'} \rangle = \mathbb{U}'$ and $\mathbb{K} \xrightarrow{\mathbf{m}^\perp}_1 \langle \langle \text{return } A \mid \gamma_{\mathbb{K}}(c_0) \mid \Sigma_{\mathbb{K}} \rangle; \gamma_{\mathbb{K}}; \emptyset \rangle = \mathbb{J}$ with $\pi_{\mathbb{U}'} = \pi'_1 \oplus \dots \oplus \pi'_p$.

First, it is clear that $r_{\mathbb{W}'} \not\approx f'_{\mathbb{U}}$, $r_{\mathbb{W}'} \not\approx f'_{\mathbb{W}}$. Second, by synchronicity, we have $F(\gamma_{\mathbb{K}}(c_\ell)) \simeq_{\phi_{\mathbb{W}}} \pi_\ell$ and $F(c_\ell[S_\ell]) \simeq_{\phi_{\mathbb{U}}} \pi'_\ell$, then if we take, for all $\ell \geq 0$, $c'_\ell = d_{\ell+1}$ and write $\gamma_{\mathbb{K}}(c_0) = c'_0[S'_0]$, then for all $\ell \geq 0$, we get $F(c'_\ell[S'_\ell]) \simeq_{\phi_{\mathbb{W}'}} \pi_\ell$ and $F(\gamma_{\mathbb{I}}(c'_\ell)) \simeq_{\phi_{\mathbb{U}'}} \pi'_{\ell+1}$, i.e. $\langle \mathbb{J}, \mathbb{W}' \rangle \mathcal{S} \langle \mathbb{J}, \mathbb{U}' \rangle$ since $\pi_{\mathbb{W}} = \pi_{\mathbb{W}'}$ and $\pi_{\mathbb{U}'} = \pi'_1 \oplus \dots \oplus \pi'_p$.

Case $\mathbf{m} = \kappa.\langle \square[\text{ret } A] \mid d_0 \rangle^\oplus$.

In this case, S_0 must be of the shape $S[\kappa[\square]]$, $\mathbf{t} = \text{return } V$, $\xi_{\mathbb{I}} = \emptyset$ and $\eta_{\mathbb{W}} = \emptyset$ and

$\mathbb{I}, \mathbb{W} \xrightarrow{\mathbf{m}} \langle \Sigma_{\mathbb{I}}; \gamma_{\mathbb{I}}; [d_0 \mapsto c_0[S]]; \emptyset \rangle, \langle [d_0] \oplus \pi_{\mathbb{W}} \mid \emptyset \mid \phi_{\mathbb{W}} \rangle$

Since $\langle \mathbb{I}, \mathbb{W} \rangle \mathcal{S} \langle \mathbb{K}, \mathbb{U} \rangle$, then $F(c_0[S[\kappa[\square]]]) \simeq_{\phi_{\mathbb{U}}} \pi'_0$ and $\xi_{\mathbb{K}} = \emptyset$.

Subcase $F(c_0[S \circ \kappa]) = \pi'_0$.

There exists $\pi''_0 = F(c_0[S])$ s.t. $\pi'_0 = \kappa :: \pi''_0$,

We have $\mathbb{U} \xrightarrow{\mathbf{m}^\perp}_{wb} \langle \pi''_0 \uparrow \pi'_{\mathbb{U}} \mid \emptyset \mid \phi_{\mathbb{U}} \rangle = \mathbb{U}'$ that is $\pi_{\mathbb{U}'} = \pi''_0 \oplus \pi'_1 \oplus \dots \oplus \pi'_p$ and $\mathbb{K} \xrightarrow{\mathbf{m}^\perp}_1 \langle \langle \text{return } A \mid d_0[\gamma_{\mathbb{K}}(\kappa)] \mid \Sigma_{\mathbb{K}} \rangle; \gamma_{\mathbb{K}}; \emptyset \rangle = \mathbb{J}$.

Subcase $F(c_0[S \circ \kappa]) \neq \pi'_0$.

There exists $f \in \phi_{\mathbb{U}}$ s.t. $F(c_0[S \circ \kappa]) = f \uparrow \pi'_0$.

We have $\mathbb{U} \xrightarrow{\mathbf{m}^\perp}_{wb} \langle \pi''_0 \uparrow \pi'_{\mathbb{U}} \mid \emptyset \mid \phi_{\mathbb{U}} \rangle = \mathbb{U}'$ that is $\pi_{\mathbb{U}'} = \pi''_0 \oplus \pi'_1 \oplus \dots \oplus \pi'_p$ where $f = \kappa :: f'$ and

$\pi''_0 = f' \uparrow \pi'_0$ and $\mathbb{K} \xrightarrow{\mathbf{m}^\perp}_1 \langle \langle \text{return } A \mid d_0[\gamma_{\mathbb{K}}(\kappa)] \mid \Sigma_{\mathbb{K}} \rangle; \gamma_{\mathbb{K}}; \emptyset \rangle = \mathbb{J}$.

In either subcase, let's write $d_0[S'_0]$ for $d_0[\gamma_{\mathbb{K}}(\kappa)]$ and, for all $\ell \geq 0$, $S''_{\ell+1} = S'_\ell$.

First, it is clear that $r_{\mathbb{W}'} \not\approx f'_{\mathbb{U}}$, $r_{\mathbb{W}'} \not\approx f'_{\mathbb{W}}$ and that $F(d_0[S'_0]) \simeq_{\phi_{\mathbb{W}'}} [d'_0]$ and that $F(\gamma_{\mathbb{I}}(d_0)) \simeq_{\phi_{\mathbb{U}'}} \pi''_0$, since $\phi_{\mathbb{U}} = \phi_{\mathbb{U}'}$, $\phi_{\mathbb{W}} = \phi_{\mathbb{W}'}$ and $\gamma_{\mathbb{J}}(d_0) = \gamma_{\mathbb{I}}(d_0) = c_0[S]$.

Then we have, by synchronicity, for all $\ell \geq 0$, $F(\gamma_{\mathbb{K}}(c_\ell)) \simeq_{\phi_{\mathbb{W}}} \pi_\ell$ and $F(c_\ell[S_\ell]) \simeq_{\phi_{\mathbb{U}}} \pi'_\ell$, that is, for all $\ell \geq 1$, $F(d_\ell[S'_\ell]) \simeq_{\phi_{\mathbb{W}'}} \pi_\ell$ and $F(\gamma_{\mathbb{I}}(d_\ell)) \simeq_{\phi_{\mathbb{U}'}} \pi'_\ell$,

Thus we can conclude that $\langle \mathbb{J}, \mathbb{W}' \rangle \mathcal{S} \langle \mathbb{J}, \mathbb{U}' \rangle$.

Case $\mathbf{m} = \mathbf{c}_0.\langle \kappa :: r[e] \mid \square \rangle^\oplus$.

In this case, $\langle \mathbf{t} \mid c_0[S_0] \circ S \rangle$ must be of the shape $\langle \underline{e} \mid c_0[\square] \circ T \oplus S \rangle$,

Subcase $\underline{e} = \iota \# \text{op } V$.

$\mathbb{I}, \mathbb{W} \xrightarrow{\mathbf{m}} \langle \Sigma_{\mathbb{I}}; \gamma_{\mathbb{I}}; \langle \mathbf{e}, \kappa :: r, \delta_{\mathbb{I}} \cdot [\kappa \mapsto T] \rangle \rangle, \langle \pi_{\mathbb{W}} \mid (f_{\mathbb{W}} \uparrow [\kappa], \mathbf{e}) \mid \phi_{\mathbb{W}} \rangle$.

Since $\langle \mathbb{I}, \mathbb{W} \rangle \mathcal{S} \langle \mathbb{K}, \mathbb{U} \rangle$, then $F(c_0[\square]) \simeq_{\phi_{\mathbb{U}}} \pi'_0$, i.e. $\pi'_0 = [c_0]$ and $\xi_{\mathbb{K}} = \langle \mathbf{e}, r, \delta_{\mathbb{K}} \rangle$.

We have then $\mathbb{K} \xrightarrow{\mathbf{m}^\perp}_1 \langle \langle \mathbf{e} \mid \gamma_{\mathbb{K}}(c_0) \circ \kappa[r\{\delta_{\mathbb{K}}\}] \mid \Sigma_{\mathbb{K}} \rangle; \gamma_{\mathbb{K}}; \langle \mathbf{e}, \kappa :: r, \delta_{\mathbb{K}} \rangle \rangle = \mathbb{J}$ and $\mathbb{U} \xrightarrow{\mathbf{m}^\perp}_{wb} \langle \pi'_1 \oplus \dots \oplus \pi'_p \mid (f_{\mathbb{U}}, \mathbf{e}) \mid \phi_{\mathbb{U}} \rangle = \mathbb{U}'$

By synchronicity, we have $F(\gamma_{\mathbb{K}}(c_\ell)) \simeq_{\phi_{\mathbb{W}}} \pi_\ell$ and $F(c_\ell[S_\ell]) \simeq_{\phi_{\mathbb{U}}} \pi'_\ell$, $r \not\approx \iota f_{\mathbb{W}}$ and $r \not\approx \iota f_{\mathbb{U}}$ then if we take, for all $\ell \geq 0$, $c'_\ell = d_{\ell+1}$ and write $\gamma_{\mathbb{K}}(c_0) = c'_0[S'_0]$, we get $\kappa :: r \not\approx \iota f'_{\mathbb{U}}$, $\kappa :: r \not\approx \iota f'_{\mathbb{W}}$ and for all $\ell \geq 0$ we get $F(c'_\ell[S'_\ell]) \simeq_{\phi_{\mathbb{W}'}} \pi_\ell$ and $F(\gamma_{\mathbb{I}}(c'_\ell)) \simeq_{\phi_{\mathbb{U}'}} \pi'_{\ell+1}$, i.e. $\langle \mathbb{J}, \mathbb{W}' \rangle \mathcal{S} \langle \mathbb{J}, \mathbb{U}' \rangle$ since $\pi_{\mathbb{W}} = \pi_{\mathbb{W}'}$ and $\pi_{\mathbb{U}'} = \pi'_1 \oplus \dots \oplus \pi'_p$.

Subcase $\underline{e} = \mathbf{e}$. $\mathbb{I}, \mathbb{W} \xrightarrow{\mathbf{m}} \langle \Sigma_{\mathbb{I}}; \gamma_{\mathbb{I}}; \langle \mathbf{e}, \kappa :: r, \delta_{\mathbb{I}} \cdot [\kappa \mapsto T] \rangle \rangle, \langle \pi_{\mathbb{W}} \mid (f_{\mathbb{W}} \uparrow [\kappa], \mathbf{e}) \mid \phi_{\mathbb{W}} \rangle$.

Since $\langle \mathbb{I}, \mathbb{W} \rangle \mathcal{S} \langle \mathbb{K}, \mathbb{U} \rangle$, then $F(c_0[\square]) \simeq_{\phi_{\mathbb{U}}} \pi'_0$, i.e. $\pi'_0 = [c_0]$ and $\xi_{\mathbb{K}} = \langle \mathbf{e}, r, \delta_{\mathbb{K}} \rangle$.

We have then $\mathbb{K} \xrightarrow{\mathbf{m}^\perp}_1 \langle \langle \mathbf{e} \mid \gamma_{\mathbb{K}}(c_0) \circ \kappa[r\{\delta_{\mathbb{K}}\}] \mid \Sigma_{\mathbb{K}} \rangle; \gamma_{\mathbb{K}}; \langle \mathbf{e}, \kappa :: r, \delta_{\mathbb{K}} \rangle \rangle = \mathbb{J}$ and $\mathbb{U} \xrightarrow{\mathbf{m}^\perp}_{wb} \langle \pi'_1 \oplus \dots \oplus \pi'_p \mid (f_{\mathbb{U}}, \mathbf{e}) \mid \phi_{\mathbb{U}} \rangle = \mathbb{U}'$

By synchronicity, we have $F(\gamma_{\mathbb{K}}(c_\ell)) \simeq_{\phi_{\mathbb{W}}} \pi_\ell$ and $F(c_\ell[S_\ell]) \simeq_{\phi_{\mathbb{U}}} \pi'_\ell$, $r \not\approx \iota f_{\mathbb{W}}$ and $r \not\approx \iota f_{\mathbb{U}}$ then if we take, for all $\ell \geq 0$, $c'_\ell = d_{\ell+1}$ and write $\gamma_{\mathbb{K}}(c_0) = c'_0[S'_0]$, we get $\kappa :: r \not\approx \iota f'_{\mathbb{U}}$, $\kappa :: r \not\approx \iota f'_{\mathbb{W}}$ and for all $\ell \geq 0$ we get $F(c'_\ell[S'_\ell]) \simeq_{\phi_{\mathbb{W}'}} \pi_\ell$ and $F(\gamma_{\mathbb{I}}(c'_\ell)) \simeq_{\phi_{\mathbb{U}'}} \pi'_{\ell+1}$, i.e. $\langle \mathbb{J}, \mathbb{W}' \rangle \mathcal{S} \langle \mathbb{J}, \mathbb{U}' \rangle$ since $\pi_{\mathbb{W}} = \pi_{\mathbb{W}'}$ and

$$\pi_{U'} = \pi'_1 \oplus \dots \oplus \pi'_p.$$

Case $\mathbf{m} = \kappa.\langle \Box[r[e]] \mid d_0 \rangle^\oplus$. Similar reasoning applies. □

Corollary E.11 *Given $\mathbb{J} \in \text{Confs}_{\text{pas}}$, $\mathbb{I} \in \text{Confs}_{\text{act}}$, $\mathbb{W}, \mathbb{U} \in \mathcal{W}$ and a trace $\mathbf{t}$ such that $\langle \mathbb{J}, \mathbb{W} \rangle \mathcal{S} \langle \mathbb{I}, \mathbb{U} \rangle$, $\mathbf{t} \in \text{Tr}_{\mathcal{L}_{\text{AI}}}(\mathbb{I})$ and $\text{Tr}_{\mathcal{L}_{\text{AI}}}(\mathbb{J})$ then:*

$$\mathbf{t} \in \text{Tr}_{\mathcal{L}_{\text{wb}}}(\mathbb{W}) \wedge \mathbf{t}^\perp \in \text{Tr}_{\mathcal{L}_{\text{wb}}}(\mathbb{U})$$

Proof. By induction on the length of $\mathbf{t}$ using Lemma E.10. □

Definition E.12 (Composability) Given a passive configuration $\mathbb{G} \in \mathcal{O}^\ominus$ and an active one $\mathbb{H} \in \mathcal{O}^\oplus$ such that $\mathbb{G} = \langle \mathbb{I} \parallel \mathbb{T} \parallel \mathbb{W} \rangle$ and $\mathbb{H} = \langle \mathbb{J} \parallel \mathbb{S} \parallel \mathbb{U} \rangle$, we say that $\mathbb{G}$ and $\mathbb{H}$ are *closed composable* and write $\mathbb{G} \perp \mathbb{H}$ when:

$$\frac{\mathbb{G} = \langle \mathbb{I} \parallel \mathbb{T} \parallel \mathbb{W} \rangle \quad \mathbb{H} = \langle \mathbb{J} \parallel \mathbb{S} \parallel \mathbb{U} \rangle \quad \langle \mathbb{I}, \mathbb{W} \rangle \mathcal{S} \langle \mathbb{J}, \mathbb{U} \rangle}{\mathbb{G} \perp \mathbb{H}}$$

Remark E.13 The preservation of *closed composability* by configuration synchronization follows from lemmas B.2 and E.10.

E.3.2 OGS Observation

Now we can semantically capture $\mathbf{E}[\mathbf{t}\{v\}] \Downarrow_{\text{op}} ()$ by the following predicate.

Definition E.14 (Observation predicate) Taking a passive configuration $\mathbb{G} \in \mathcal{O}^\ominus$ and an active one $\mathbb{H} \in \mathcal{O}^\oplus$ such that $\mathbb{G} \perp \mathbb{H}$, we define the observation of $\mathbb{G}$ on $\mathbb{H}$ as the predicate:

$$\langle \mathbb{G} \perp \mathbb{H} \rangle \Downarrow [c] \text{ret} () : \iff \exists \mathbb{K}, \gamma. \langle \mathbb{J} \Vdash \mathbb{I} \rangle \Downarrow_{\text{ci}} \langle \mathbb{K} \Vdash c[\text{return} ()]; \gamma; \perp \rangle$$

The soundness of this observation predicate follows from Theorem C.1, Lemmas B.2, E.3, E.10. Its completeness is a consequence of the following result: the definability of configurations that are composable with an initial active configuration.

Lemma E.15 (OGS definability) *Taking an initial active OGS configuration $\mathbb{G} = \langle \Sigma; \Gamma \vdash d[\mathbf{t}] : \perp \rangle_{\text{ogs}}$, then for all passive OGS configuration $\mathbb{H}$ such that $\mathbb{G} \perp \mathbb{H}$, there exist:*

- a continuation name: c_f
- an evaluation context: $c_f : \neg\sigma \vdash K : \neg\tau$,
- a name valuation: $\Sigma'; \emptyset \vdash v : \Gamma$,

such that $\mathbb{H} = \langle \Sigma'; c_f : \neg\sigma \vdash [d \mapsto K] \cdot v \rangle_{\text{ogs}}$

Proof. We will write $\mathbb{G} = \langle \mathbb{I} \parallel \mathbb{T} \parallel \mathbb{W} \rangle$ with $\mathbb{I} = \langle (d[\mathbf{t}] \mid \Sigma); \varepsilon; \emptyset \rangle$, $\mathbb{T} = \langle \Gamma; \emptyset \mid \emptyset; \emptyset \rangle$ and $\mathbb{W} = \langle [] \mid \emptyset \mid \emptyset \rangle$. We will also write $\mathbb{H} = \langle \mathbb{J} \parallel \mathbb{S} \parallel \mathbb{U} \rangle$ with $\mathbb{J} = \langle \Sigma'; \gamma; \xi \rangle$ and $\mathbb{U} = \langle \pi \mid \eta \mid \phi \rangle$.

By complementarity, we have (1) $\text{supp}(d[\mathbf{t}]) \subseteq \text{dom}(\gamma)$, and (2) there exist a type σ and continuation name $c_f \in \text{codom}(\gamma)$ s.t. $\mathbb{S} = \langle [c_f \mapsto \neg\sigma]; \emptyset \Vdash \Gamma; \emptyset \rangle$.

(1) ensures the existence of a name valuation v s.t. $\text{dom}(v) = \text{supp}(\mathbf{t})$ and an interactive context K s.t. $\gamma = [d \mapsto K] \cdot v$, whereas (2) entails $\Sigma'; [c_f \mapsto \neg\sigma] \vdash \gamma : \Gamma$ and $\text{dom}(\gamma) = \text{dom}(\Gamma)$.

$\Sigma'; c_f : \neg\sigma \vdash K : \Gamma(d)$ and $\Sigma'; \emptyset \vdash v : \Gamma$.

By compatibility of configurations $\mathbb{J}$ and $\mathbb{U}$, we have $\text{supp}(\phi) \subseteq \text{dom}(v) \cap \mathcal{K}$, and since $\text{dom}(v) = \text{supp}(\mathbf{t})$ and $\text{supp}(\mathbf{t}) \cap \mathcal{K} = \emptyset$, then $\phi = \emptyset$.

By synchronicity, we have $F(\gamma(d)) \simeq_\emptyset \pi$; that is $F(K) = [c_f] = \pi$, and we also have $\xi = \emptyset$, which again by compatibility entails that $\emptyset \sim_{\text{fwd}} \eta$ and $\eta = \emptyset$.

Finally, we have shown that $\mathbb{H} = \langle \langle \Sigma'; \gamma; \emptyset \rangle \parallel \langle [c_f \mapsto \neg\sigma]; \emptyset \Vdash \Gamma; \emptyset \rangle \parallel \langle [c_f] \mid \emptyset \mid \emptyset \rangle \rangle$, i.e. $\mathbb{H} = \langle \Sigma'; c_f : \neg\sigma \vdash [d \mapsto K] \cdot v \rangle_{\text{ogs}}$. □

Corollary E.16 (completeness of observation) *Taking two initial active OGS configurations $\mathbb{G}_1 = \langle \Sigma; \Gamma \vdash_c M_1 \rangle_{\text{ogs}}$ and $\mathbb{G}_2 = \langle \Sigma; \Gamma \vdash_c M_2 \rangle_{\text{ogs}}$ such that $M_1 \simeq_{\text{ciu}} M_2$, then:*

$$\begin{aligned} & \forall H \in \mathcal{O} \text{ s.t. } H \perp \mathbb{G}_1 \text{ and } H \perp \mathbb{G}_2. \\ & \langle H \perp \mathbb{G}_1 \rangle \Downarrow [c]\text{ret}() \text{ if and only if } \langle H \perp \mathbb{G}_2 \rangle \Downarrow [c]\text{ret}() \end{aligned}$$

E.4 Interpretation of observation

We have given a sensible notion of observation at the level of OGS configurations, it remains to show what does this predicate mean at the level of OGS interpretations.

Theorem E.17 (full observation) *Given two composable configurations $\mathbb{G}$ and $\mathbb{H}$, we have:*

$$\langle H \perp \mathbb{G} \rangle \Downarrow [c]\text{ret}() \text{ if and only if } \exists t \in \mathbf{CTr}_{\text{ogs}}(\mathbb{G}). t^\perp \mathbf{c}.\langle \text{ret}() \mid \square \rangle^\oplus \in \mathbf{CTr}_{\text{ogs}}(\mathbb{H})$$

Proof. We write $\mathbb{G} = \langle I \parallel T \parallel W \rangle$ and $\mathbb{H} = \langle J \parallel S \parallel U \rangle$ and $I = \langle (t \mid \Sigma); \gamma_I; \xi \rangle$

Left to right. Suppose there exist $\mathbb{K}, \mathbb{J} \in \mathcal{A}$ and γ such that $\langle J \vdash I \rangle \Downarrow_{\text{ci}} \langle \mathbb{K} \vdash J \rangle$ with $J = \langle (c[\text{return}()] \mid \Sigma); \gamma; \emptyset \rangle$.

By corollary B.5 and definition B.3, there exists an odd-length trace t such that $I \xRightarrow{t}_1 \mathbb{K}$ and $J \xRightarrow{t^\perp}_1 \mathbb{J} \xrightarrow{\mathbf{c}.\langle \text{ret}() \mid \square \rangle^\oplus} \langle \gamma; \emptyset \rangle = J'$. By lemma E.3, there exist $T', S', S'' \in \mathcal{T}$ s.t. $T \xrightarrow{t}_T T', S \xrightarrow{t^\perp}_T S' \xrightarrow{\mathbf{c}.\langle \text{ret}() \mid \square \rangle^\oplus} S'', \mathbb{K} \circ T', J' \circ S'$ and $J' \circ S''$. Similarly, corollary E.11 ensures the existence of $W', U', U'' \in \mathcal{W}$ s.t. $W \xrightarrow{t}_{wb} W', U \xrightarrow{t^\perp}_{wb} U' \xrightarrow{\mathbf{c}.\langle \text{ret}() \mid \square \rangle^\oplus}_{wb} U''$ whereas lemma E.10 ensures that synchronicity is preserved and that in particular $\langle J, U' \rangle \mathcal{S} \langle \mathbb{K}, W' \rangle$.

Thus we have shown that $\mathbb{G} \xRightarrow[t_{\text{ogs}}]{t} \langle \mathbb{K} \parallel T' \parallel W' \rangle$ and $\mathbb{H} \xRightarrow[t_{\text{ogs}}]{t^\perp} \langle J \parallel S' \parallel U' \rangle \xrightarrow[t_{\text{ogs}}]{\mathbf{c}.\langle \text{ret}() \mid \square \rangle^\oplus} \langle J' \parallel S'' \parallel U'' \rangle$, from which we can deduce that $t \in \mathbf{Tr}_{\text{ogs}}(\mathbb{G})$ and $t^\perp \mathbf{c}.\langle \text{ret}() \mid \square \rangle^\oplus \in \mathbf{Tr}_{\text{ogs}}(\mathbb{H})$.

Right to left. Let $t \in \mathbf{Tr}_{\text{ogs}}$ such that $t \in \mathbf{Tr}_{\text{ogs}}(\mathbb{G})$ and $t^\perp \mathbf{c}.\langle \text{ret}() \mid \square \rangle^\oplus \in \mathbf{Tr}_{\text{ogs}}(\mathbb{H})$.

If we write $\mathbb{G} \xRightarrow[t_{\text{ogs}}]{t} \langle \mathbb{K} \parallel T' \parallel W' \rangle$ and for some $W' \in \mathcal{W}$ with $\mathbb{K} = \langle \Sigma_{\mathbb{K}}; \gamma_{\mathbb{K}}; \emptyset \rangle \in \mathcal{A}$, and write $\mathbb{H} \xRightarrow[t_{\text{ogs}}]{t^\perp} \langle J \parallel S' \parallel U' \rangle \xrightarrow[t_{\text{ogs}}]{\mathbf{c}.\langle \text{ret}() \mid \square \rangle^\oplus} \mathbb{E}$ for some $S' \in \mathcal{T}, U' \in \mathcal{W}$ and $\mathbb{E} \in \mathcal{O}$ with $J = \langle (c[\text{return}()] \mid \Sigma); \gamma; \emptyset \rangle$, then by corollary B.5, we have $\langle J \vdash I \rangle \mapsto^* \langle J \vdash \mathbb{K} \rangle$ and again by lemma B.2, we have $J \vdash_c \mathbb{K}$, and in particular $c \notin \text{dom}(\gamma_J)$, thus $\langle J \vdash \mathbb{K} \rangle \not\Downarrow_{\text{ci}}$. Finally, we have shown that $\langle J \vdash I \rangle \Downarrow_{\text{ci}} \langle J \vdash \mathbb{K} \rangle$ and consequently $\langle H \perp \mathbb{G} \rangle \Downarrow [c]\text{ret}()$. □